



ANALYSIS REPORT

25612-EXE.r1.v1 NUMBER

Malware Analysis Report

2025-06-12 DATE

Notification

- Author : github.com/miho030
- TLP Classification : TLP:WHITE
- Purpose : Personal development and public information sharing (non-commercial)

This report is the result of independent research and self-development by an individual malware analyst. It is shared for the benefit of the cybersecurity community and the public good. This document is provided "as-is" information purpose only. All content reflects the author's technical opinion at the time of analysis and **does not represent the official position of any specific organization, institution, or government agency**. No warranties of any kind are expressed or implied regarding the contents of this report. Please note that some of the information in this document (eg., malware hashes, C2 addresses) may be security-sensitive. Caution is advised when testing or applying this data to live systems. All responsibility for the use of this material rests solely with the user.

You are free to use this report for the following purposes, as long as the information is not misused and proper attribution is provided:

- Educational or research purpose
- Threat intelligence sharing
- Reference material for malware response and defense

The format of this report is based on and adapted from the (public domain docs) TLP:WHITE malware analysis reports published by *America's Cyber Defense Agency (CISA)*. I'm hereby clearly state that this report is not affiliated with CISA, *Department of Homeland Security (DHS)*, or any government entity. The non-commercial reuse of this format is permitted under the following legal and policy bases:

- [CISA - Traffic Light Protocol \(TLP\)](#)
"Subject to standard copyright rules, TLP:WHITE information may be distributed without restriction."
- [U.S Copyright Office - 17 U.S.C. § 105](#)
"Copyright protection under this title is not available for any work of the United States Government."
- [FIRST.org - TLP](#)
"TLP:CLEAR information may be shared without restriction"

However, if you create derivative works based on this report, you must include this 'Notification' section with proper attribution to the original author and source.

Summary

Description

Amadey Bot first appeared around 2018 and is being distributed by Russian-speaking dark web forums in the form of Malware-as-a-Service (MaaS).

Below are typical features of Amadey Bot.

- Information stealer
Collect browser login information, clipboard data, system/AV information, etc. and sends it to the C2 server
- Downloader Role
Download additional malicious payload from C2 server (ex : LockBit ransomware)
- BotNet Features
It is a structure that can perform DDoS attacks by connecting infected devices to botnet.

Technical features include anti-sandbox processing, infection persistence process, and plugin based expansion. Amadey Bot checks the execution path and attempts to avoid analysis by allowing code execution only in a specific temporary folder. In addition, through the registry, start folder, and scheduler registration, Amadey Bot is set to operate even after rebooting. It tries to disable security countermeasures by changing the permissions of files it stores using trusted programs such as calcs and making it difficult to delete. It has the characteristic of modularizing clipboard and credential takeover through plug-ins such as clip64.dll and credit64.dll.

On March 22, 2023, [DarkTrace](#) reviewed Amadey Bot that exploited the N-Day vulnerability. It is being used in variants such as MaaS models and SmokeLoader and continues to spread.

On December 7, 2023, [Cyver Solutions by Thales](#) reviewed Amadey Bot is also mentioned in reviews of supply-side attacks by high-end chip particles such as LockBit 3.0 ransomware and Secret Blizzard.

Ahnlab's ASEC team wrote a report that a spear-fishing attack technology was used in the process of installing Amadey Bot as a module of LockBit 3.0 ransomware.

As such, Amadey Bot-type malware continues to be used as a favorite tool by attackers.

Submitted Files (1)

449d9e29d49dea9697c9a84bb7cc68b50343014d9e14667875a83cade9adbc60 (sample.exe)

Dropped Files (1)

449d9e29d49dea9697c9a84bb7cc68b50343014d9e14667875a83cade9adbc60 (oneetx.exe)

IPs (2)

77.91.124.145

77.91.124.207

Findings

449d9e29d49dea9697c9a84bb7cc68b50343014d9e14667875a83cade9adbc60

Tags

Amadey-bot trojan stealer spy-ware exe

Details

Verdict

Malware activity

File name

449d9e29d49dea9697c9a84bb7cc68b50343014d9e14667875a83cade9adbc60.exe

File size

1.04 MB (1090048 bytes)

File type

Win32 EXE

Magic

PE32 executable (GUI) Intel 80386, for MS Windows

MD5

26b31e11da1e9fb60b64b91414884eb9

SHA256

449d9e29d49dea9697c9a84bb7cc68b50343014d9e14667875a83cade9adbc60

ssdeep

24576:nyCRSPtqsGuWIIQEsWkMTuFha/Ygj1XQK0/ClkRUONndOC8PLf:yC4F5rWIIQEs5MTuM3jlqlrnde8P

Entropy

59/71 (VT)

Packers/Compilers/Cryptos

Packer : Compressed or packed data[High entropy + Section 3 (".rsrc") compressed]

Compiler : Microsoft Visual C/C++(19.13.26213)[LTCG/C]

Crypto : n/a

Antivirus

Ahnlab	Trojan/Win.Avkiller.C5408483
Alyac	Spyware.Infostealer.RedLine
Avast	Win32:PWSX-gen [Trj]
Avira	TR/Spy.Stealer.vauxz
Bitdefender	Trojan.GenericKD.66331193
Fortinet	W32/Kryptik.HTGV!tr
Kaspersky	UDS:Trojan-PSW.Win32.Stealerc.gen
MacAfee	Ti!449D9E29D49D
Tencent	Trojan.Win32.Kryptik.hr
symantec	ML.Attribute.HighConfidence

YARA Rules

Portable Executable Info

Header

Dos Header	MZ (DOS Signature)
NT Header Signature	PE (NT Signature)
Entry Point	27232 : 0x00006A60 (section: .text)
Number of sections	5
Optional Header	GUI
Debug Data	Binary[Offset=0x0874,Size=0x25], PDB file link(7.0)

Contained Sections

PE Sections

Name	Virtual Address	VirtSize (byte)	RawSize (byte)	Entropy	MD5
.text	4096	25364	25600	6.314	b0b66b32f4ca82e2e157c51b24da0be7
.data	32768	6728	512	4.971	7b9890a93c0516bb070e1170cfde54d5
.idata	40960	4178	4608	5.026	67ce48bf2e7c8fe3321ca7aa188f77e2
.rsrc	40960	1056768	1055744	7.960	67ce48bf2e7c8fe3321ca7aa188f77e2
.reloc	1105920	2184	2560	6.223	67ce48bf2e7c8fe3321ca7aa188f77e2

PE Metadata (EXIF)

PEType	PE32 (32-bit)
Code size	0010f000
Import Hash	186AC8D51CD85A98171D94FDE8CAC1AD
Compiler TimeStamp	Tue May 24 22:49:06 2022 (UTC)
Created time	n/a
Modified time	n/a
InitializedDataSize	1063424 bytes
UninitializedDataSize	0 bytes
Compiler	Microsoft Visual C/C++(19.13.26213)[LTCG/C]
Tool	Visual Studio(2017, v15.6)
LinkerVersion	Microsoft Linker 14.13
OSVersion	Windows 11
File OS	NT Windows32 [offset:001c]
ImageVersion	10.0
Subsystem	GUI
Production name	Internet Explorer
Company name	Microsoft Corporation
Internal name	Wextract
Original File name	WEXTRACT.EXE .MUI
Product version	11.00.17763.1 [offset: 0010, 0014]
File version	11.00.17763.1 (WinBuild.160101.0800) [offset: 0008, 000c]
File description	Win32 Cabinet Self-Extractor
Legal Copyright	LegalCopyright,?Microsoft Corporation. All rights reserved.
File flags	n/a
Language code	0x0409 (English-US)
Code page(Char-Set)	1200 (Unicode UTF-16, little endian)

Imports

ADVAPI32.dll
KERNEL32.dll
GDI32.dll
USER32.dll
msvcrt.dll
COMCTL32.dll
Cabinet.dll
VERSION.dll

Findings

449d9e29d49dea9697c9a84bb7cc68b50343014d9e14667875a83cade9adbc60

Tags

- Amadey-bot
- trojan
- stealer
- spy-ware
- exe

Details

Verdict	Malware activity
File name	oneetx.exe
File size	235.50 KB (241152 bytes)
File type	Win32 EXE
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
MD5	b0ec0839deec888969ebc4cf6d8a378
SHA256	38cdabe18a8fe52499bd387b7b3bd328d3927805be5b61d2971245a11851c78f
ssdeep	3072:liirj6QA6XZcVA7DDMIbJ1j8WyLsLP829t5vs/WC/W8JCQbu2mSLT/C5Qkf:iiG6Euvvg7gWyLUP59tKtb17bv8y
Entropy	62 / 72 (VT)

Packers/Compilers/Cryptos

packer : (Heur) Compressed or packed data[Section 0 (" .text") compressed]

Antivirus

Ahnlab	Trojan/Win.Avkiller.C5408483
Alyac	Spyware.Infostealer.RedLine
Avast	Win32:MalwareX-gen [Pws]
Avira	TR/Spy.Stealer.vauxz
Bitdefender	Trojan.GenericKD.66331193
Fortinet	W32/Kryptik.HTGV!tr
Kaspersky	UDS:Trojan-PSW.Win32.Stealerc.gen
MacAfee	Ti!449D9E29D49D
Tencent	Trojan.Win32.Kryptik.hr
symantec	ML.Attribute.HighConfidence

YARA Rules

Portable Executable Info

Header

Dos Header	MZ (DOS Signature)
NT Header Signature	PE (NT Signature)

Entry Point	0x00005137 (section: .text)
Number of sections	4
Optional Header	GUI
Debug Data	Binary[Offset=0x0874,Size=0x25], PDB file link(7.0)

Contained Sections					
PE Sections					
Name	Virtual Address	VirtSize (byte)	RawSize (byte)	Entropy	MD5
.text	4096	183428	183808	7.74	7cdf8b3ce7de13f0af9616ba968759d9
.data	188416	450136	10240	1.94	94936b1b42cc9dfa9b8dd10f997b3910
.rsrc	638976	38608	38912	4.72	3864422d93d4a5c655f0ae8f872065d6
.reloc	679936	6936	7168	3.34	97176c4e91026e291ece5d9b3cf0796e

PE Metadata (EXIF)	
PEType	PE32 (32-bit)
Code size	183808 bytes
Import Hash	8EFFD33C0C7D3209E4D0DDDE05803B138
Compiler TimeStamp	Wed Apr 13 19:19:55 2022
InitializedDataSize	496640 bytes
UninitializedDataSize	0 bytes
Compiler	Microsoft Visual C/C++(15.00.21022)[LTCG/C++]
LinkerVersion	Microsoft Linker(9.00.21022)
OSVersion	Windows 2000
ImageVersion	0.0
Subsystem	Windows GUI
Production name	Sheat
Company name	n/a
Internal name	TravelingTimes
Original File name	oneetx.exe
Product version	n/a
File version	n/a
File description	n/a
Legal Copyright	Copyright (C) 2023, maersk
Language code	Spanish-Bolivia neutral unknown

Imports
KERNEL32.dll
GDI32.dll
USER32.dll

Description

Analysis environment

- OS : Windows 10 (64bit) VM

Analysis Summary step

- 1. Saving the sample file and extracting the initial '.exe'
- 2. Using Entropy to identify that the initial '.exe' is packed
- 3. Using a debugger to manually unpack the first payload
- 4. Initial analysis of the unpacked payload
- 5. Identifying interesting strings and imports
- 6. Static Analysis to establish the context of interesting strings and imports
- 7. Utilising a debugger to analyse the string decryption function
- 8. Automating the string decryption using x32Dbg
- 9. Utilising decrypted string to identify the malware family
- 10. extract IoC information and make YARA-Rules

First, Analyst tried upload sample at Detect-it-easy for check packing or common file information.

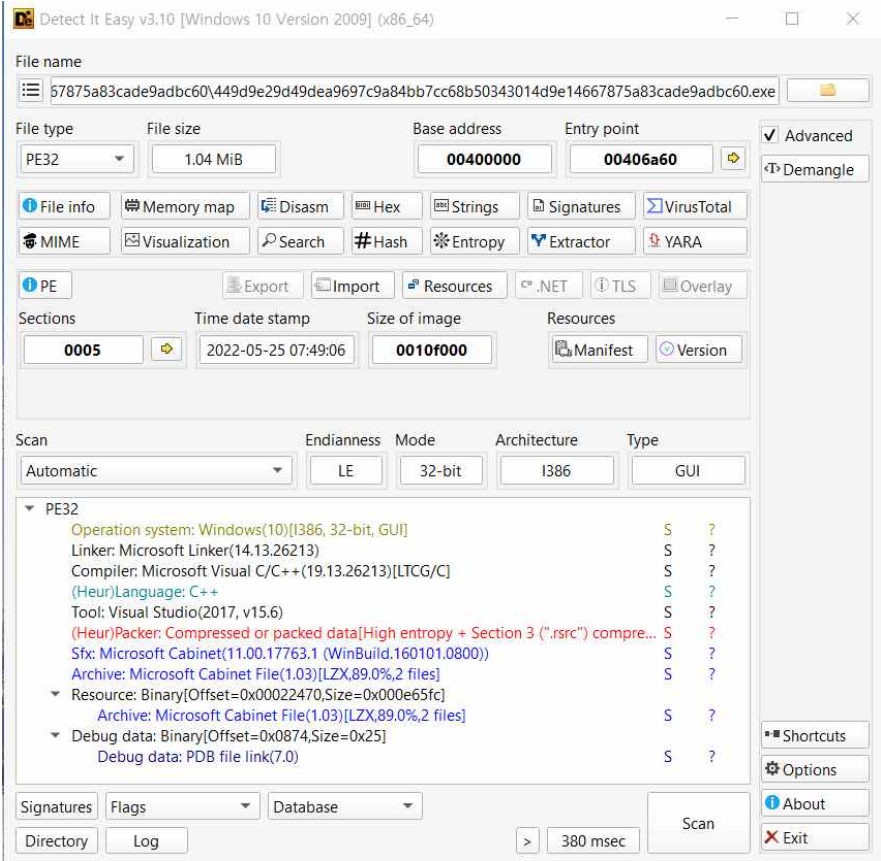


Figure 1 - Initial Malware analysis using Detect-it-easy

According to the analysis result of DIE, the file to be analyzed is a 32-bit executable file. In the case of this sample, it is actually a

[Microsoft Cabinet file](#). That means, This file essentially a '.zip' that can be executed as a '.exe' file.

The Microsoft Cabinet file is similar enough to '.zip' or '.7z'. It is able to extract the contents of the file just like a regular zip file.

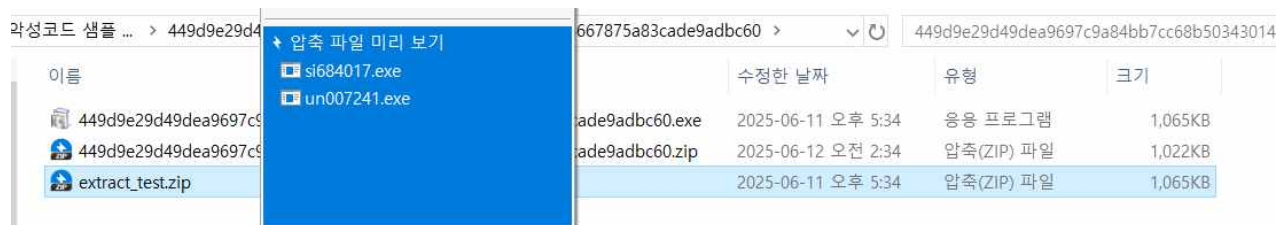


Figure 2 - 2 hidden exe file exposed by decompression program's preview function.

I was able to use 'bandi-zip' program to extract the contents from file, as you can see at 'figure-2' image, analyst confirmed 2 hidden exe files identified by the decompression program's preview function after changing the analyzed file to a zip extension. the archived file created 2 new exes in the process. These are '**si684017.exe**' and '**un007241.exe**' in the screenshot below.



Figure 3 - extracted exe file from sample file.

Analyst tried to check 'si684017.exe' file using Detect-it-easy. During initial analysis, always want to determine if the file is potentially a packed-loader rather than a final payload. if I have reason to suspect a packed payload, I typically focus on unpacking rather than strings or other static analysis.

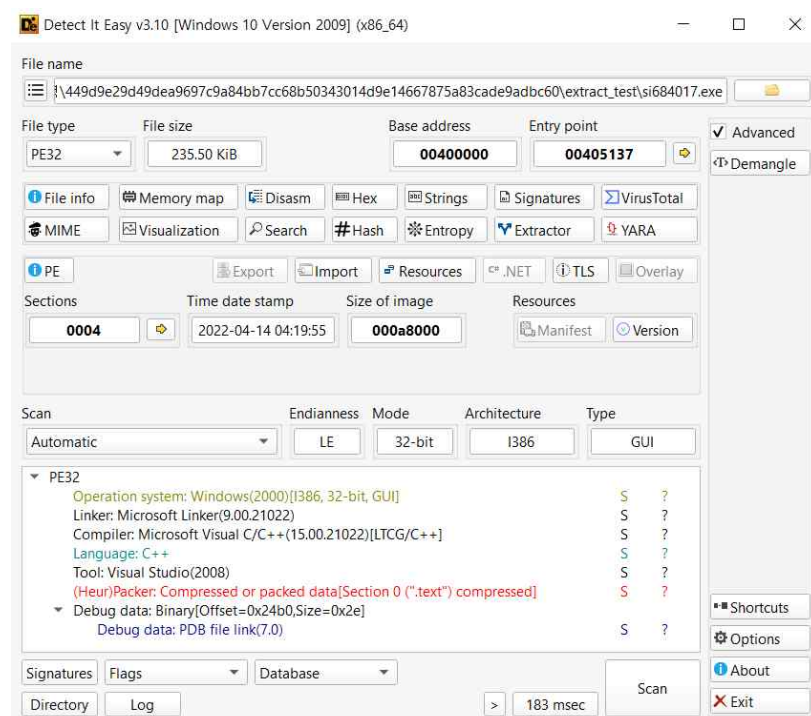


Figure 4 - Initial Malware('si684017.exe') analysis using Detect-it-easy

In commonly, packed sections have a high entropy value. To determine areas of entropy, Analyst utilized the 'Entropy Graph' feature within Detect-it-easy. Also, I found 2 sections with high entropy values. (especially '.rsrc' section.)

And also, This is a strong indication that the file is a packed loader and not the final payload.

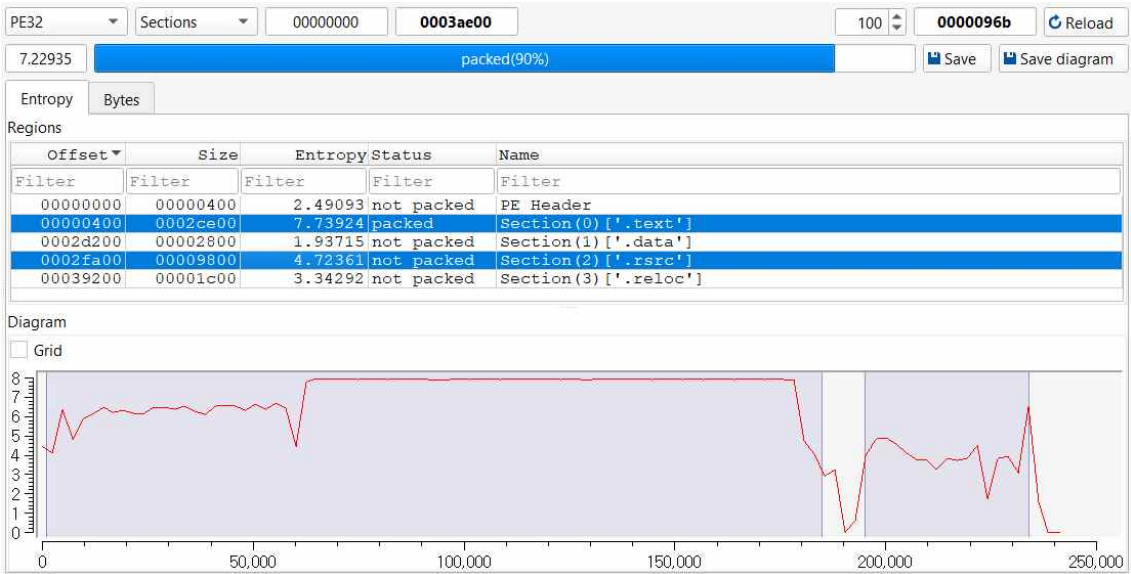


Figure 5 - Malware entropy analysis using Detect-it-easy

Unpacking Using x32Dbg

The standard loader on Windows systems typically uses combinations such as 'VirtualAlloc', 'VirtualProtect', and 'CreateThread' to decompress files. These features allow malware to allocate new memory sections that can be used to store and execute uncompressed payloads.

However, some of samples of the advanced malware type are designed to highly obfuscate or not fully use these functions. but otherwise, most of them are operated using the aforementioned functions.

In the static analysis process, a break-point may be set for most malware samples. The result is monitored by setting the break-point (and also hardware-break-point) of the x32Dbg debugger in the section where the malware sample calls the 'VirtualAlloc' and 'VirtualProtect' functions. When malware accesses the newly assigned buffer, it can receive notifications through x32Dbg, and in this process, payloads can be easily obtained without the decoded unpacking process.

I will follow right below steps in this report.

- identify a function of interest (VirtualAlloc)
- Create a break-point to monitor (VirtualAlloc)
- Obtain the memory buffer created by VirtualAlloc
- Use a Hardware break-point (to alert when the new memory buffer is accessed)
- Allow the malware to execute until the buffer is filled
- Save the buffer to a file

Loading sample file into x32Dbg

To initiate this process, you can drag the file into a x32Dbg debugger and allowed the file to execute until the Entry Point, This can be done by loading the file and just once clicking the 'F9' button. at x32Dbg.

The main purpose of 'VirtualAlloc' is to allocate memory and return the address to the newly assigned buffer. This newly assigned memory will be included in the 'EAX register' when the functionality is complete.

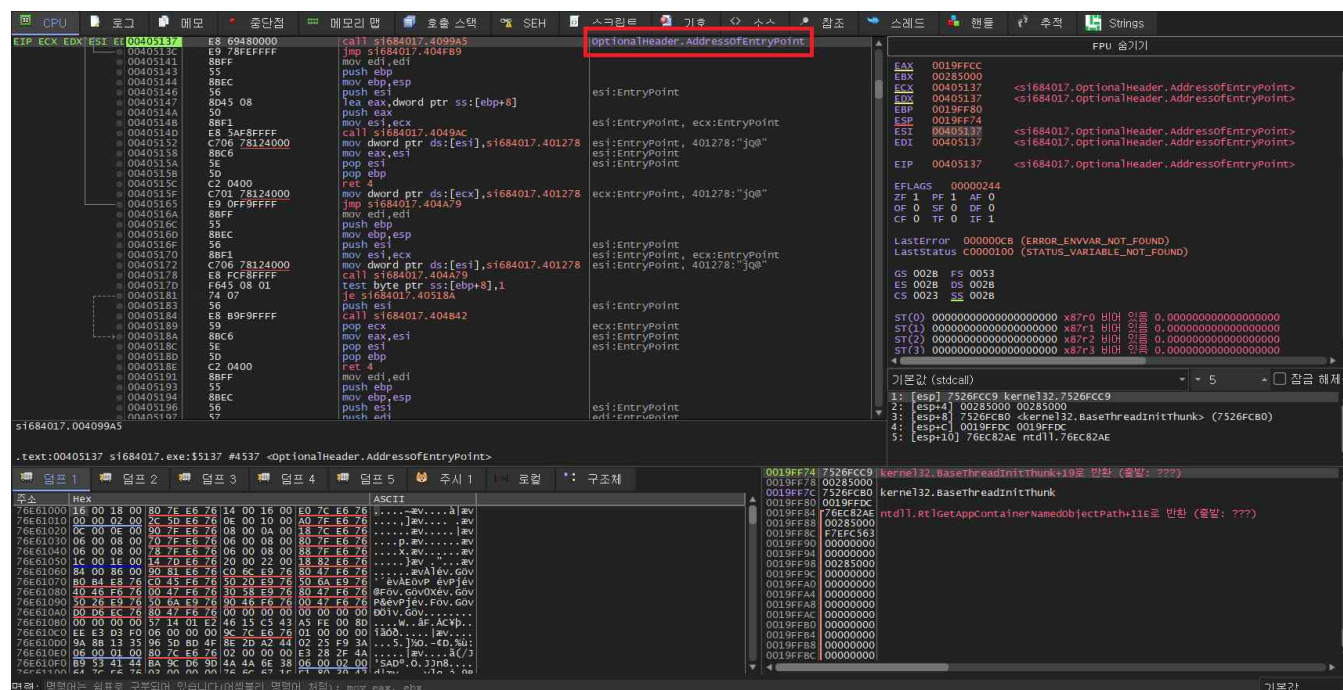


Figure 6 - Entering to malware sample's entrypoint using debugger (x32dbg)

After that, I set up breakpoint. Importantly, the result (return value) can be viewed immediately after the 'VirtualAlloc' function is called, not just before it is called.

Set breakpoints for 2 functions -> **VirtualAlloc and VirtualProtect**

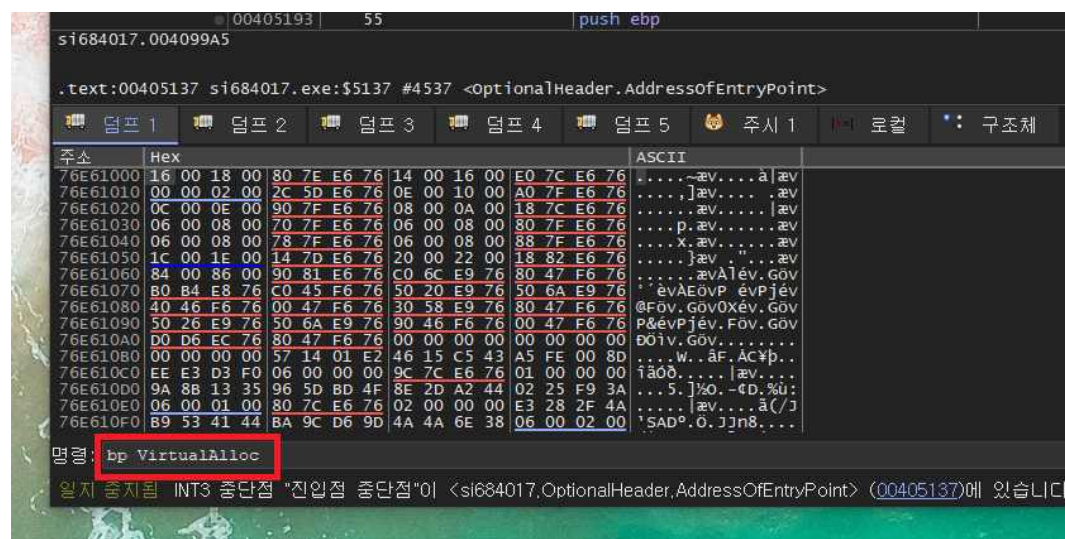


Figure 7 - setting break-point using command function in x32Dbg

The method of setting the BP(Break Point) can be set by entering it as follows in the command tab of 232Dbg.

```
bp VirtualAlloc
bp VirtualProtect
```

Move on from x32Dbg until you reach the VirtualAlloc breakpoint. The important thing is what's in that buffer. Therefore, x32Dbg's

'ctr+F9'(Execute until return) function will move right after the VirtualAlloc function ends and check the result.

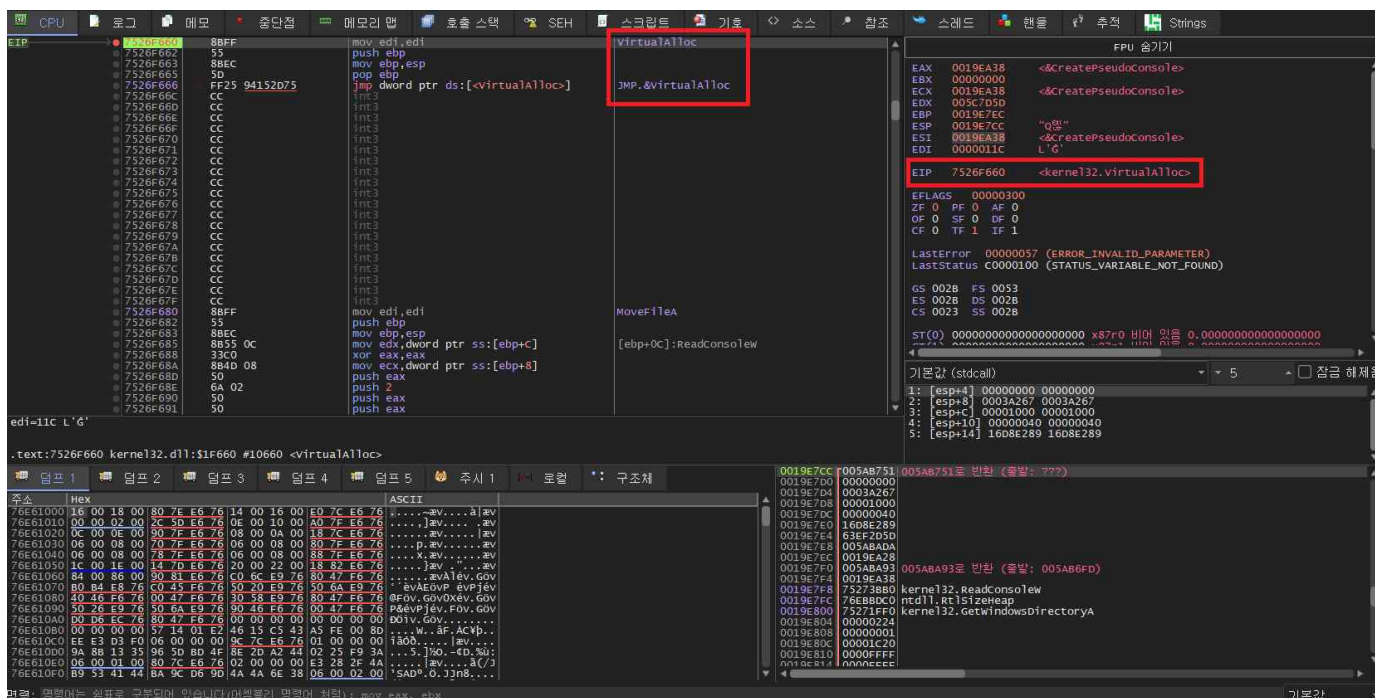


Figure 8 - Triggering breakpoint on VirtualAlloc

Allowing the malware to execute until return, it provide an 'EAX' register containing the address of the memory buffer to be used by the malware.

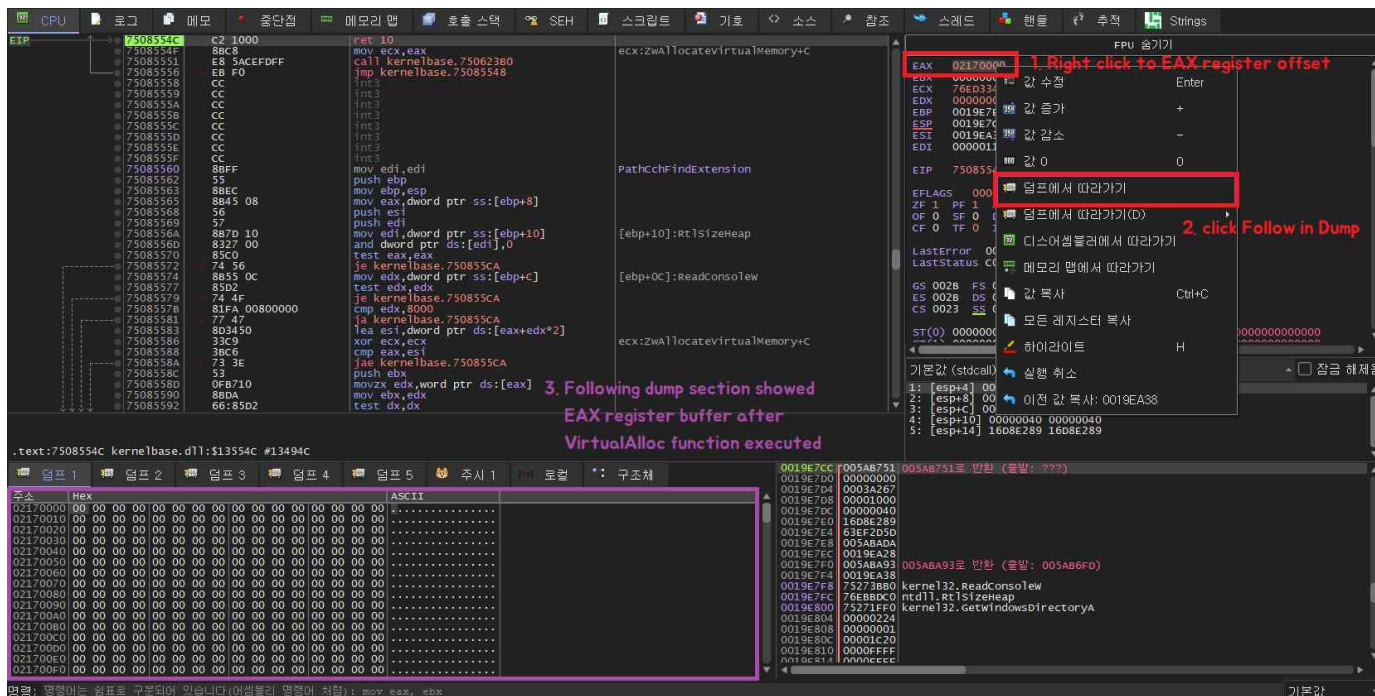


Figure 9 - Viewing the 'EAX' memory buffer returned by VirtualAlloc

The EAX register is nothing special, it is simply a standard register used to return the results of a function. To monitor the buffer returned address to select the address '0290000' and right-click to execute the 'Follow in Dump' function.



Figure 10 - Using x32Dbg to view the contents of a memory buffer

The buffer memory currently contains all '00', as nothing has been or written to the buffer yet. It is important to be notified when that buffer of '00' is no longer a buffer of '00'. To achieve this, Hardware break-point can be applied on the first byte of the newly allocated buffer.

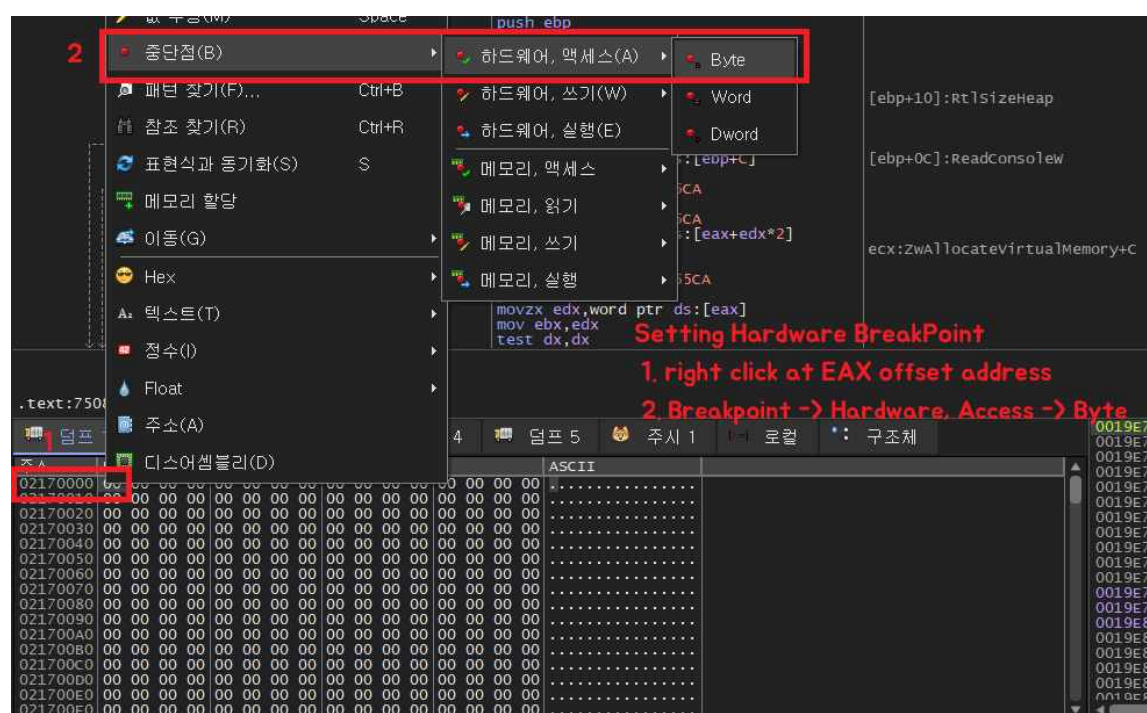


Figure 11 - Setting a Hardware Breakpoint using x32dbg

Once the hardware breakpoint is set, x32Dbg show setup info like below screenshot.

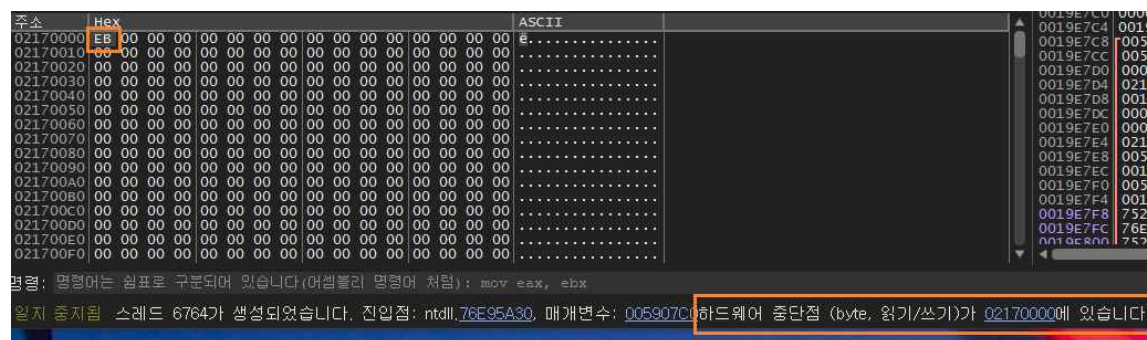


Figure 12 - Successful creation of a hardware breakpoint

in the unpacked first buffer, No PE format was found. but I think Presumably, first buffer contains a large ShellCode buffer used to decompress the next section. in the unpacked second buffer, PE format with 'MZ' header were confirmed.



Figure 13 - Identifying 2 memory buffer has Shellcode, PE format file.

The values returned after the VirtualAlloc function was called were stored in two buffers at the time of Hardware break-point. In this process, the code section in which the value of the first buffer was compressed without error was saved normally, and no type of file format can be verified, so in this case, it is likely ShellCode.

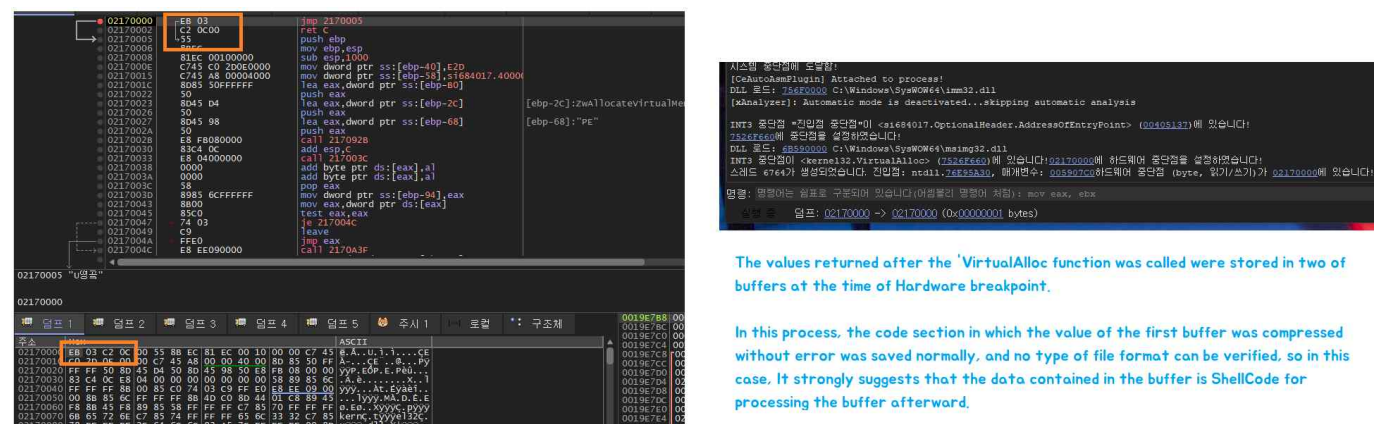


Figure 14 - identifying memory buffer containing ShellCode

Hitting F9 to allow the malware to execute, another breakpoint is hit on 'VirtualAlloc' you can see at value <kernel32.VirtualAlloc> at EIP register.

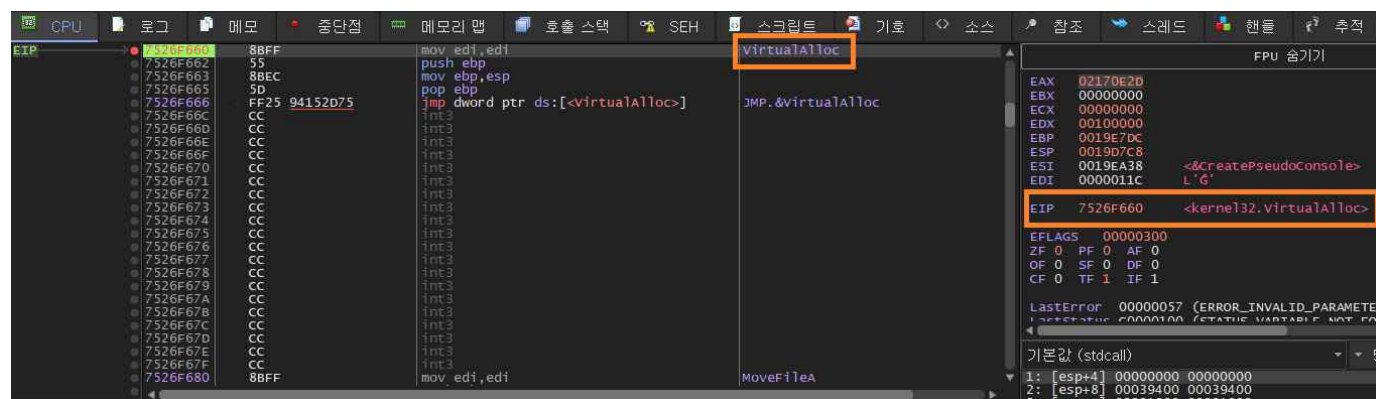


Figure 15 - viewing VirtualAlloc function in a debugger (x32Dbg)

using the same trick of Execute Until Return, Select EAX and rightclick -> Follow in dump, the second allocated buffer can be obtained. and Analyst set the hardware break-point at offset '022F000', the final unpacked pe-file payload will store that memory buffer.

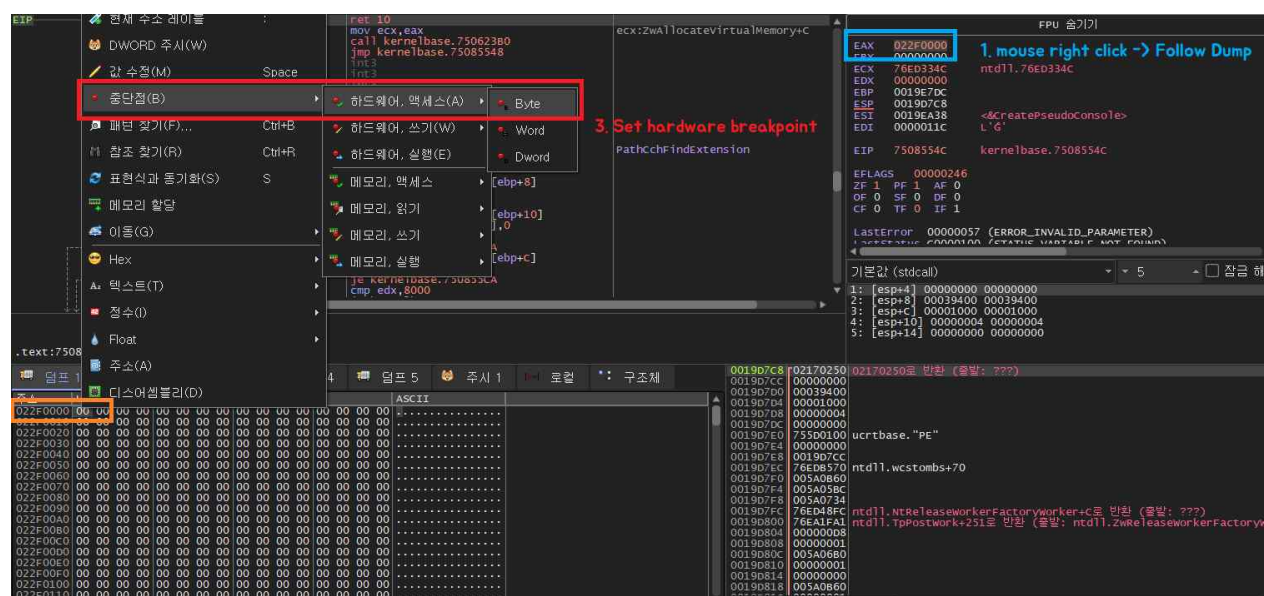


Figure 16 - Using x32Dbg to locate another memory buffer returned by VirtualAlloc

Allowing the malware to continue to execute, the hardware breakpoint is hit. this time containing a promising 'M' (First half on an MZ header)

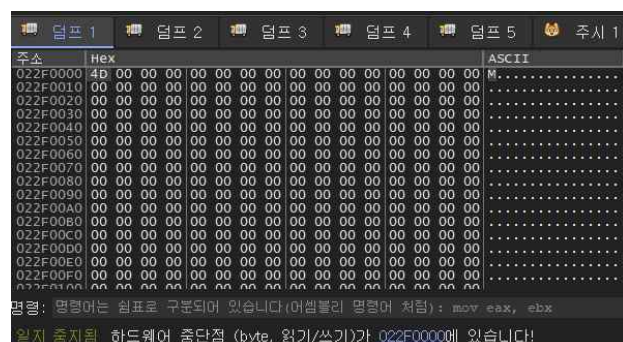


Figure 17 - Potentially containing an unpacked pe-file payload at Memory buffer

A complete MZ/PE file can be found at that memory buffer. At this point, the unpacked file has been successfully loaded into memory by allowing the malware to continue to execute. At the dump viewer, right click -> Following to memory map, and another right click at the memory map, finally can save memory buffer as a file.

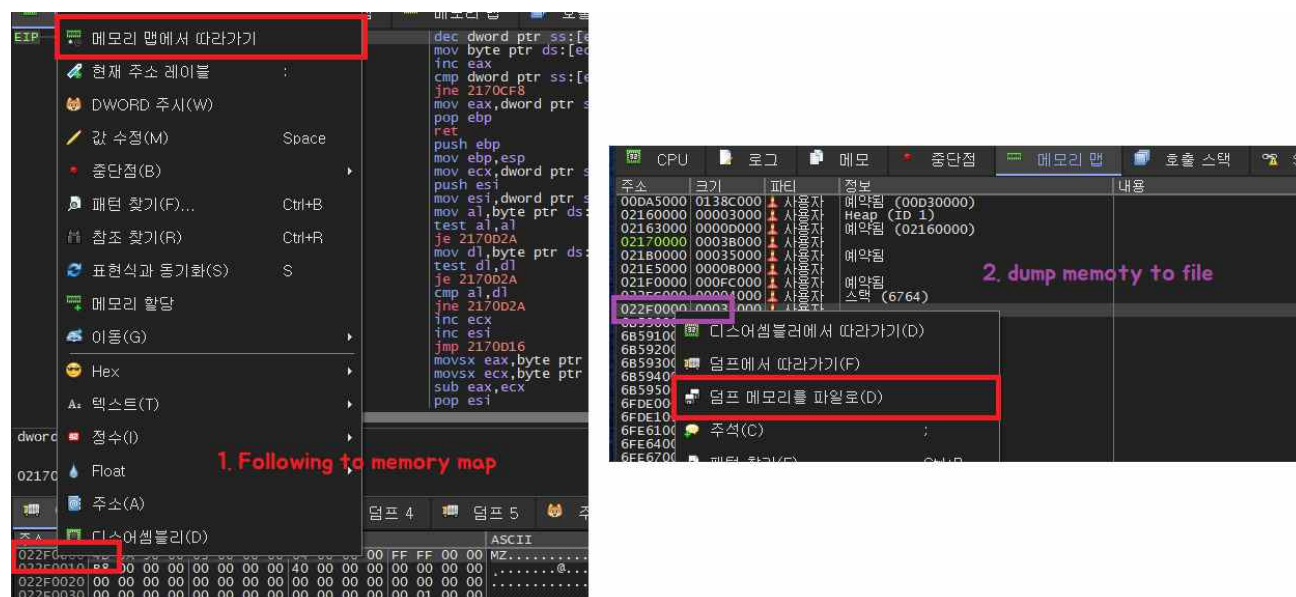


Figure 18 - how to save a memory buffer using x32Dbg

It can be create 2 binary files through the process in Figure18. In the process of writing this report, 2 file were extracted, unzipped payload dump and unzipped pe-file payload.



Figure 19 - final unzipped payload list

The file has a PE format of 32-bit and does not say whether it is packed or not, unlike when I first analyzed the sample. This process confirmed that the '.text.' section was completely unpacked now.

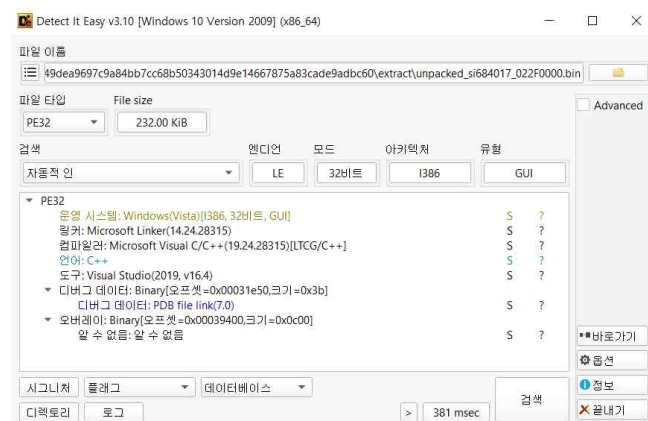


Figure 20 - Initial analysis of suspected unpacked payload using Detect-It-Easy

Entropy graphs do not contain regions with significantly higher entropy or flatness. This suggests that the files are unpackaged and contain no additional payloads.

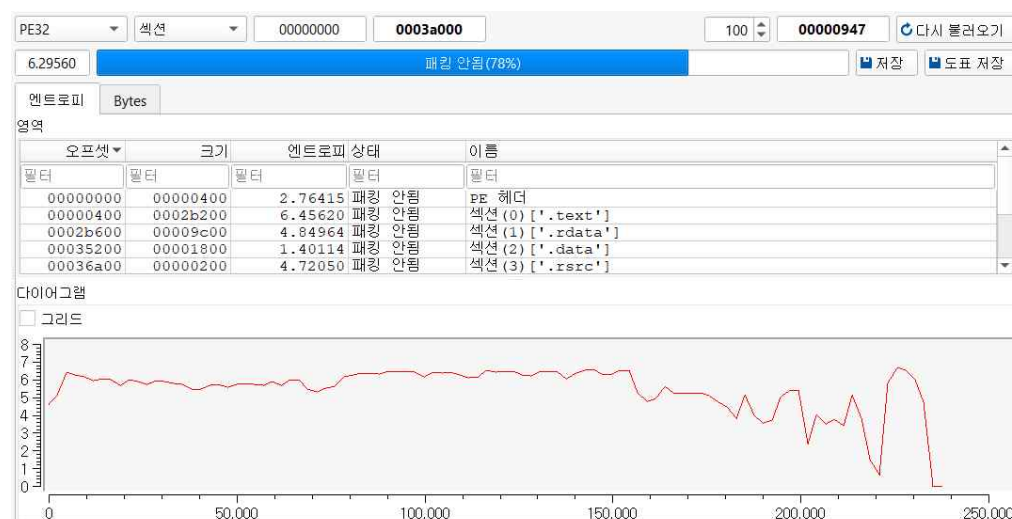


Figure 21 - Entropy analysis table

Figure 22 below is the result of checking the import table of the unpacked payload. In this process, it was found that the malicious code includes the function of downloading additional files from the outside by calling WinAPI functions such as 'HTTPOpenRequestA' and 'InternetReadFile'.

해시 64

해시 32

0000003f9df91a52

6af40414

저장

#	OriginalFirstThunk	TimeDateStamp	ForwarderChain	Name	FirstThunk	이름
필터	필터	필터	필터	필터	필터	필터
0	00035f70	00000000	00000000	00036...	0002d024	c94fe... KERNEL32.dll
1	00035f4c	00000000	00000000	00036...	0002d000	94d39... ADVAPI32.dll
2	0003611c	00000000	00000000	00036...	0002d1d0	e4b9f... KERNEL32.dll
3	00036130	00000000	00000000	00036...	0002d1e4	85b05... WININET.dll

저장

#	Thunk	Ordinal	Hint	이름
필터	필터	필터	필터	필터
0	00036...		0078	HttpOpenRequestA
1	00036...		00ce	InternetReadFile
2	00036...		009b	InternetConnectA
3	00036...		007f	HttpSendRequestA
4	00036...		0095	InternetCloseHandle
5	00036...		00c6	InternetOpenA
6	00036...		00c9	InternetOpenW
7	00036...		00c7	InternetOpenUrlA

Figure 22 - import table at unpacked payload file

Since this is likely the final payload, I checked if there is any information in the string that is not confusing. The attached Figure22 at below shows each string view table of unpacked and packed file.

PE32	섹션	ANSI	Unicode	Null-terminated	유효	15	RegEx	저장	비교
Number	오프셋	주소	크기	유형	문자열	주소	크기	유형	문자열
137	000349a8	004363a8	섹션 (1)...	16 A	ConvertSidToStringSidW	137	000349a8	004363a8	섹션 (1)...
138	000349ac	004363ac	섹션 (1)...	10 A	CreateDirectoryA	138	000349ac	004363ac	섹션 (1)...
139	000349d0	004363d0	섹션 (1)...	22 A	DllGetClassObject	139	000349d0	004363d0	섹션 (1)...
140	000349d4	004363d4	섹션 (1)...	14 A	DllGetClassObject	140	000349d4	004363d4	섹션 (1)...
141	000349d8	004363d8	섹션 (1)...	15 A	DeleteCriticalSection	141	000349d8	004363d8	섹션 (1)...
142	000349dc	004363dc	섹션 (1)...	3C A	DestroyFile	142	000349dc	004363dc	섹션 (1)...
143	000349e0	004363e0	섹션 (1)...	10 A	FindFirstFileExW	143	000349e0	004363e0	섹션 (1)...
144	000349e4	004363e4	섹션 (1)...	10 A	FlushFileBuffers	144	000349e4	004363e4	섹션 (1)...
145	000349e8	004363e8	섹션 (1)...	14 A	FileTimeToSystemTime	145	000349e8	004363e8	섹션 (1)...
146	000349ec	004363ec	섹션 (1)...	10 A	FindFirstFileExW	146	000349ec	004363ec	섹션 (1)...
147	000349f0	004363f0	섹션 (1)...	14 A	FileTimeToSystemTime	147	000349f0	004363f0	섹션 (1)...
148	000349f4	004363f4	섹션 (1)...	10 A	FindFirstFileExW	148	000349f4	004363f4	섹션 (1)...
149	000349f8	004363f8	섹션 (1)...	14 A	FileTimeToSystemTime	149	000349f8	004363f8	섹션 (1)...
150	000349fc	004363fc	섹션 (1)...	10 A	FindFirstFileExW	150	000349fc	004363fc	섹션 (1)...
151	00034a00	00436400	섹션 (1)...	14 A	FileTimeToSystemTime	151	00034a00	00436400	섹션 (1)...
152	00034a04	00436404	섹션 (1)...	10 A	FindFirstFileExW	152	00034a04	00436404	섹션 (1)...
153	00034a08	00436408	섹션 (1)...	14 A	FileTimeToSystemTime	153	00034a08	00436408	섹션 (1)...
154	00034a0c	0043640c	섹션 (1)...	10 A	FindFirstFileExW	154	00034a0c	0043640c	섹션 (1)...
155	00034a10	00436410	섹션 (1)...	14 A	FileTimeToSystemTime	155	00034a10	00436410	섹션 (1)...

Figure 23 - Comparison of string view table between unpacked file and packed file.

This revealed some base64-encoded data, but I cannot to successfully decipher them. It is likely that the base64 encoding was combined with further obfuscation.

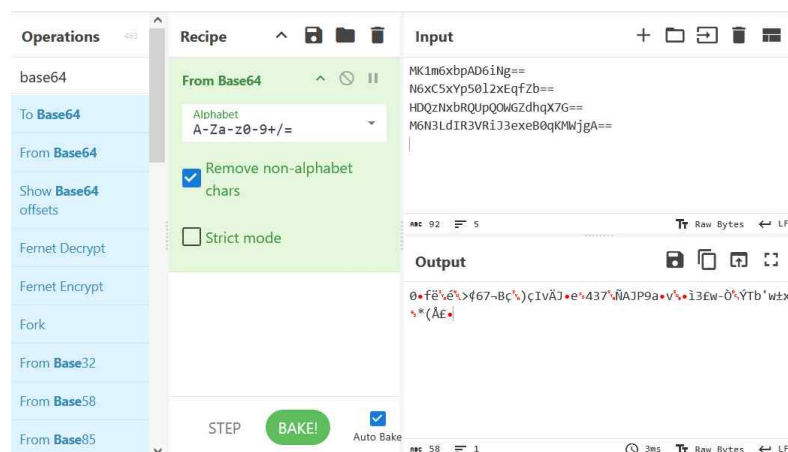


Figure 24 – failed to decode the base64 string from unpacked pe-file payload

During this search, I found an analysis report written by [Sonic Wall](#) in June 13, 2023, and a pseudocode written to implement Amadey's decoding function externally in that report.

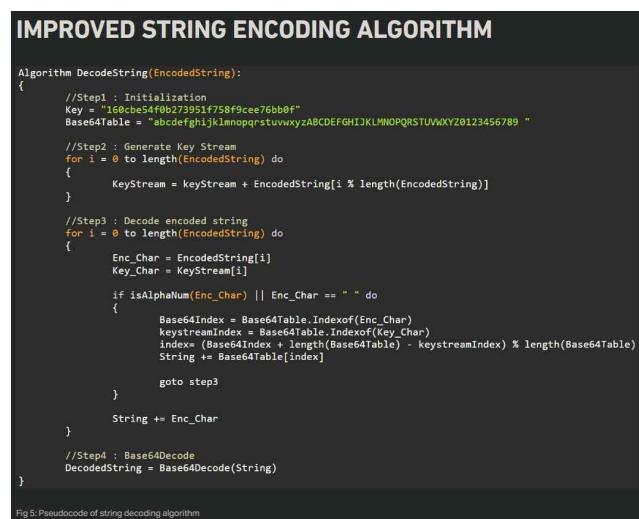


Figure 25 – pseudocode from Sonic Wall

However, as shown in Figure 25 below, this challenge failed after many attempts.

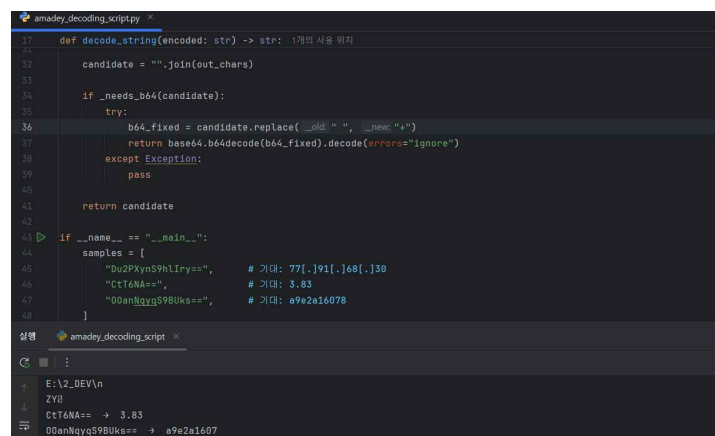


Figure 26 – failed to decode string value from unpacked payload using external script

So, I tried a different way. Amadey must contain a decoding function that can decode this string array. Therefore, I thought that if I write the value returned from this decoding function through a debugger, I would be able to get the value decoded in plaintext.

So I searched string values using Ghidra.

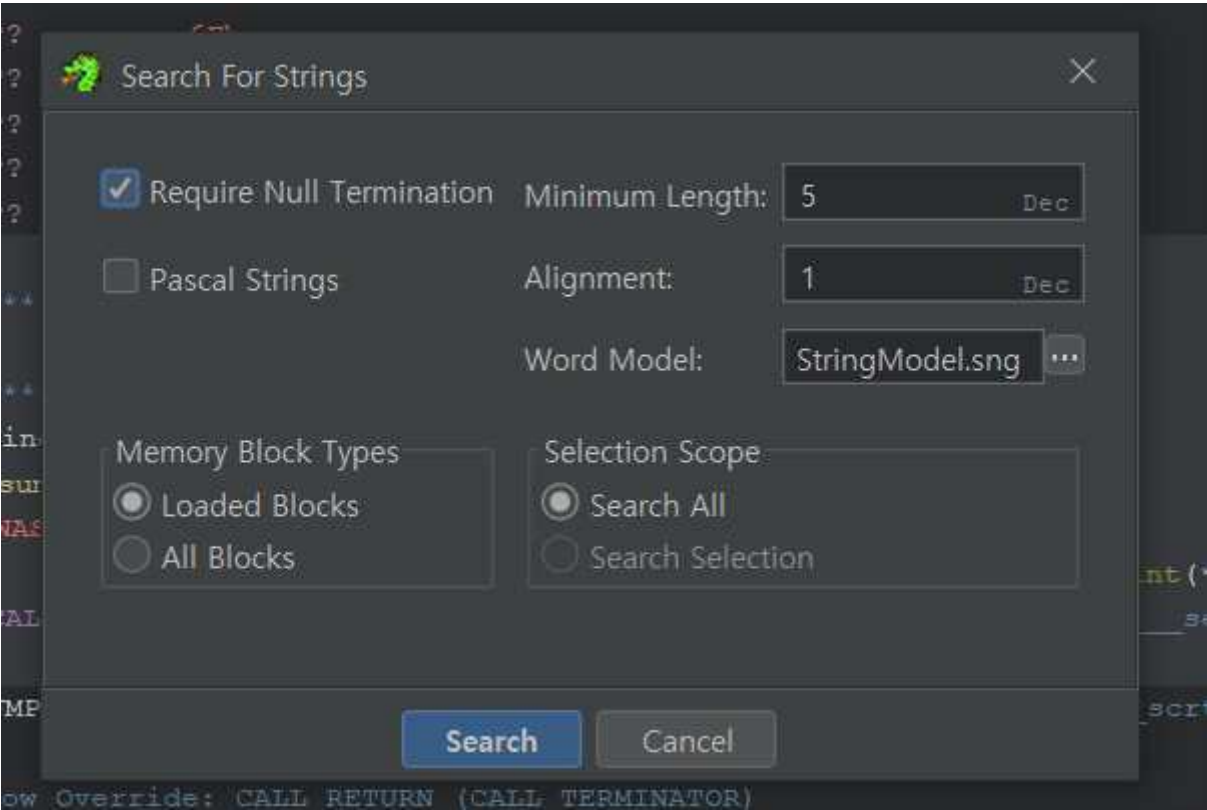


Figure 27 - searching string values using Ghidra

By filtering, we were able to quickly narrow the result down to a previously identified base64 string, which was not all related, but a solid starting point.

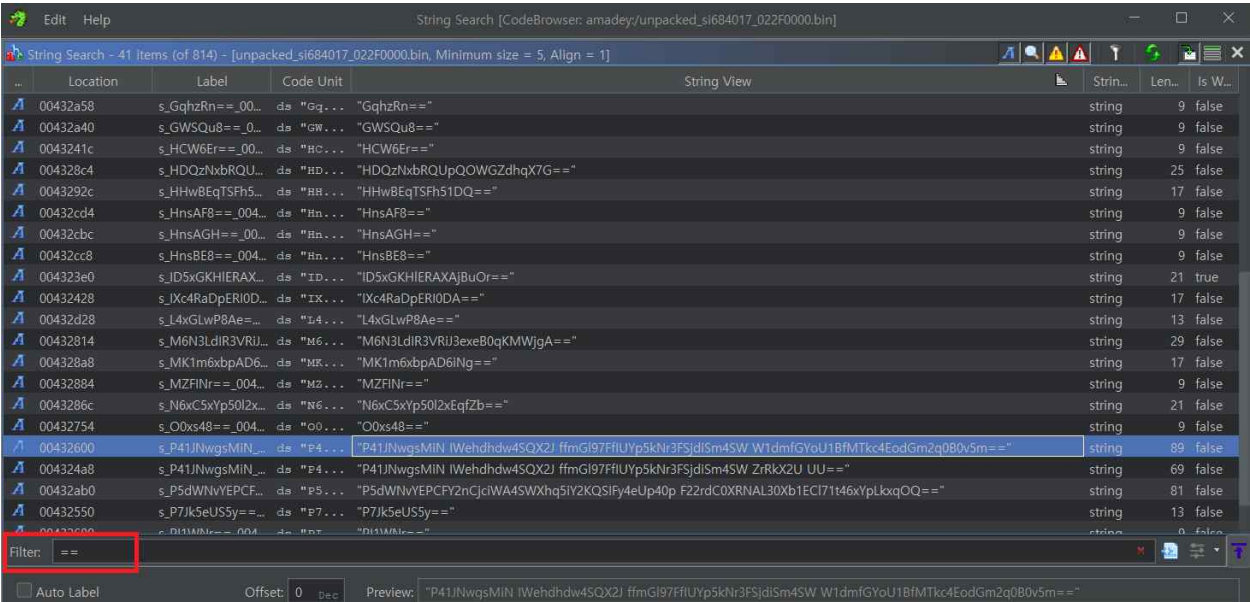


Figure 28 - finding base64 like string values using Ghidra's filter func

In Ghidra, I double-clicked a specific base64 like line output from the String viewer table to move that string to the defined offset. Using the right-click -> References -> Show References To Address function in the variable that defines the string, you can obtain the line offset address value of the function that refers to the variable. I was able to double-click this address value to check the function.

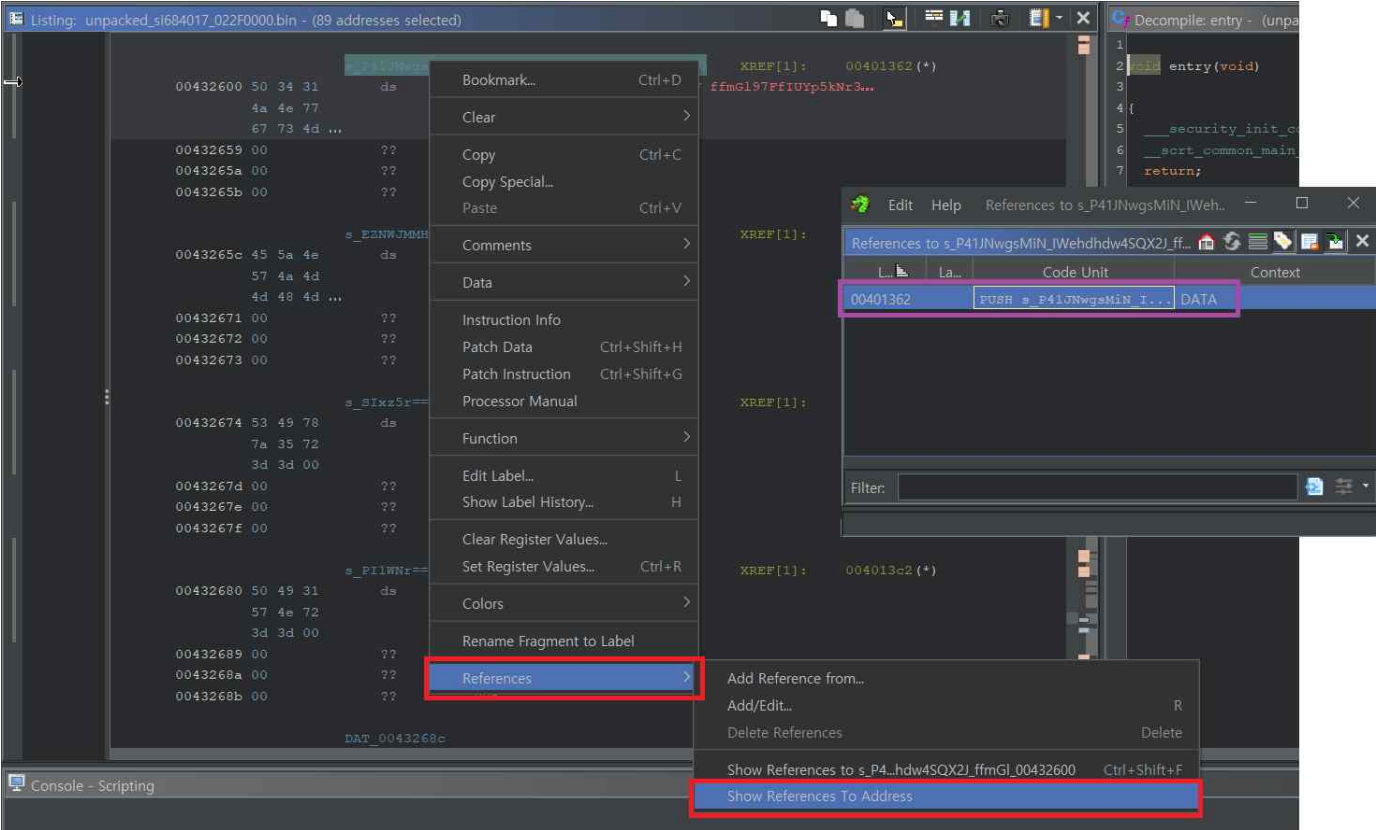


Figure 29 - Trace and locate cross-references from strings using Ghidra

Click on one of the available references to see the undefined function that acts on the string. Click on the first address of the function and press F to define the function from the current address. After defining the function, the decompiler output looks much cleaner.

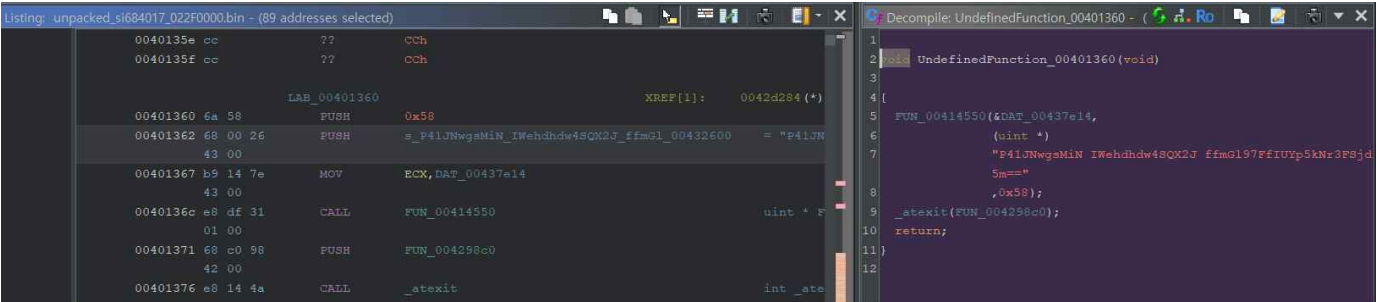


Figure 30 - Encountering an Undefined Function in Ghidra

A reference to FUN_00414550 was found in the function using the string variable. The offset address value of this function was 00414550, and it was judged to be the logic of processing a string similar to the base64 I extracted, so I tried to check it by setting it as a breakpoint.

I checked the number of cross-references on the FUN_00414550 function. A high number of cross-references would indicate that the function is responsible for decoding more than just this encoded string. If the same function is used for all string-related

decryption, then perhaps a debugger and a breakpoint are the better approach. At minimum - a debugger will at least confirm the theory that this function is related to string decryption.

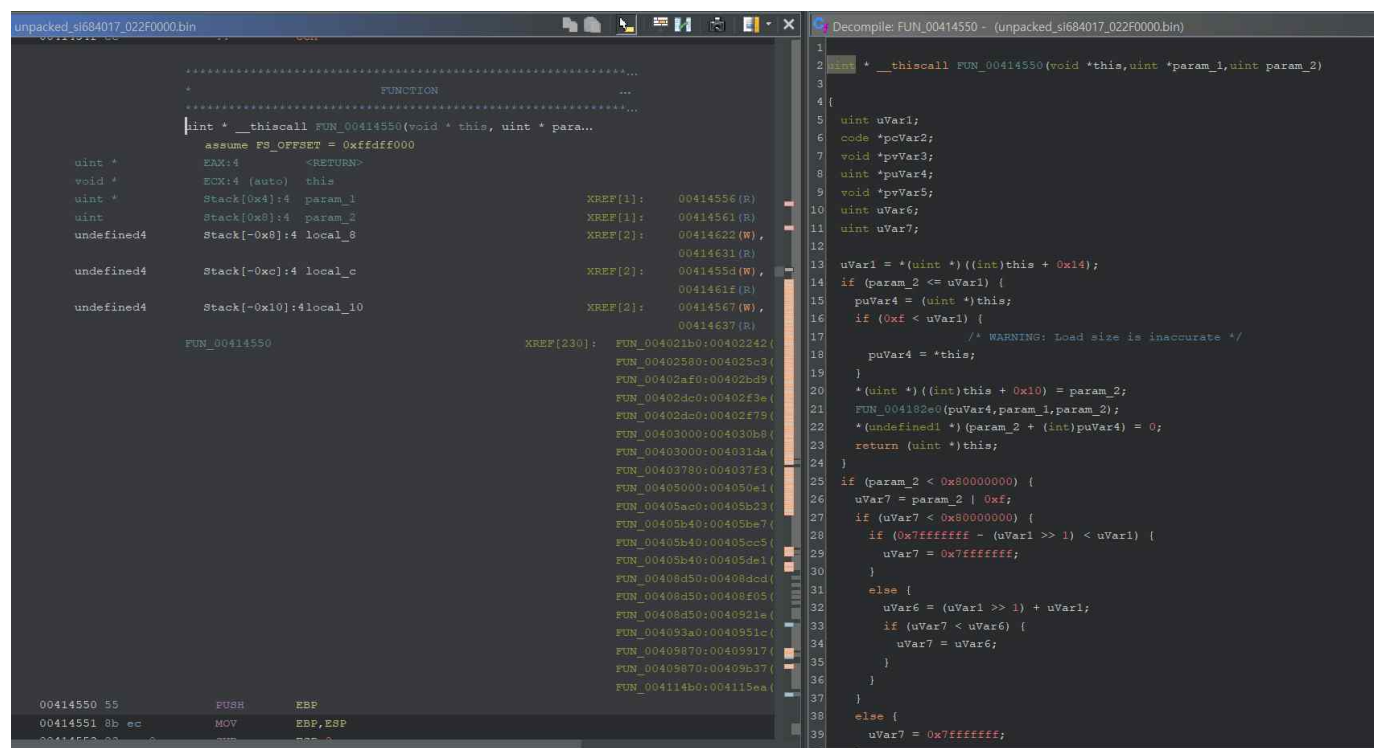


Figure 31 - analysis of FUN_00414550 function

I decided to investigate the string decryption using X32dbg. To do this, you must set a breakpoint for the function you suspect is responsible for deciphering the string. If you try to copy and paste the address directly from Ghidra, the address may not match, which is likely to result in an error. Address synchronization between Ghidra and X32dbg. Need to find the base of the current file. This can be found on the memory map, in this case it is 003e0000 which may be different.

주소	크기	파티	정보	내용	유형	보호	초기
002C0000	00001000	사용자	unpacked.bin		IMG	-R---	ERWC-
002C1000	0002C000	사용자	".text"		IMG	-R---	ERWC-
002ED000	0000A000	사용자	".rdata"		IMG	-R---	ERWC-
002F7000	00003000	사용자	".data"		IMG	-RW--	ERWC-
002FA000	00001000	사용자	".rsrc"		IMG	-R---	ERWC-
002FB000	00003000	사용자	".reloc"		IMG	-R---	ERWC-
008B0000	00010000	사용자			MAP	-RW--	-RW--
008C0000	00002000	사용자			MAP	-R---	-R---
008D0000	00002000	사용자			MAP	-R---	-R---
008E0000	0001D000	사용자			MAP	-R---	-R---
00C00000	000C5000	사용자	예약됨		PRV	-RW--	-RW--
00CC5000	0000E000	사용자	PEB, TEB (2600), wow64 TEB (2600)		PRV	-RW--	-RW--
00CD3000	0012D000	사용자	예약됨 (00C00000)		PRV	-RW--	-RW--

Figure 32 - identify base address in x32dbg debugger

The photo **Figure33** attached below explains how to synchronize Ghidra with x32dbg.

In the top tab of Ghidra, press the button of the same shape as No. 1 in the picture below to run the Ghidra Memory Map. If you click the home button in the upper right of the popped Memory Map window, a Base image address setting window will appear, where if you enter the base image address value obtained in **Figure32** and press the OK button, the address value of Ghidra will be completely synchronized with x32dbg.

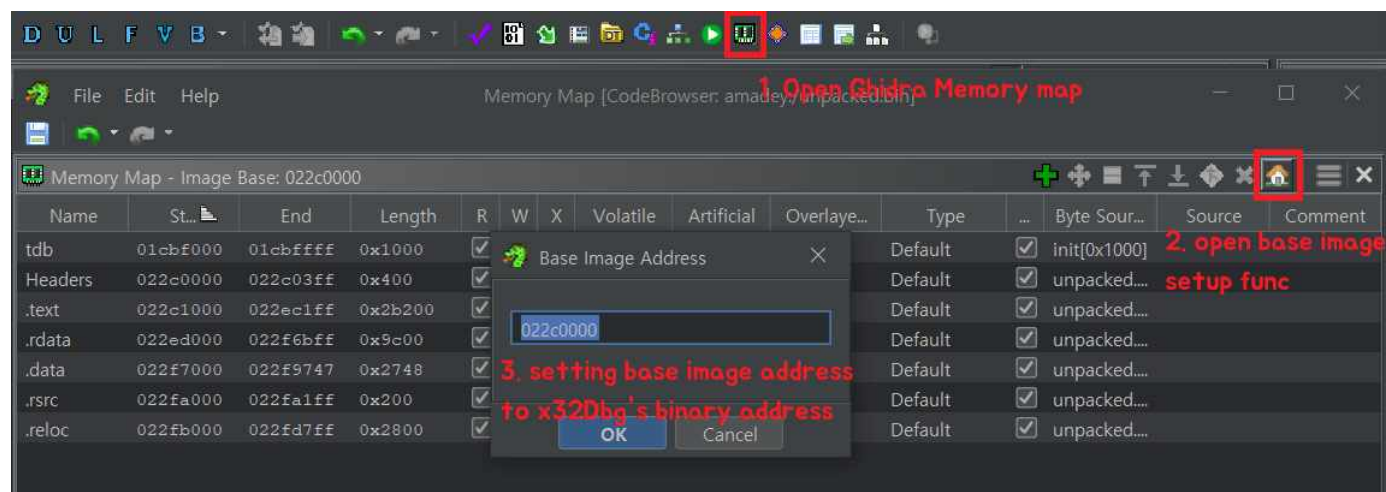


Figure 33 - Sync memory address between x32Dbg and Ghidra

After that, find the offset address value of the function of interest found in Figure 31 in Ghidra.

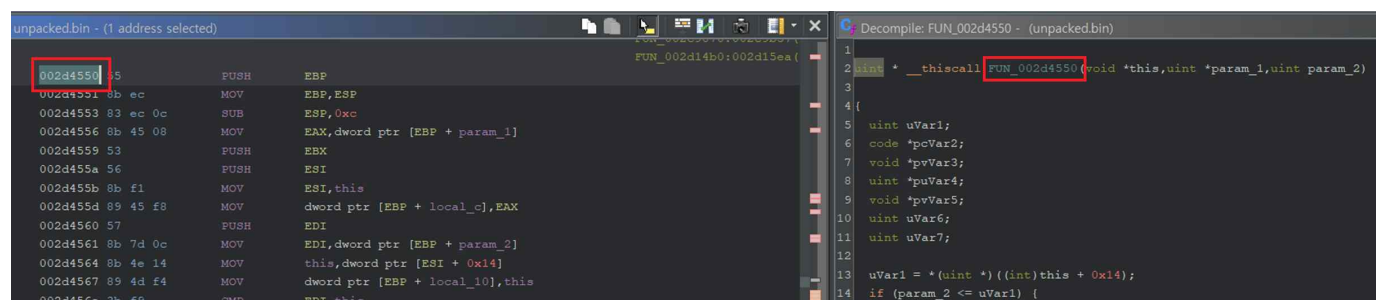


Figure 34 - find offset address what I interested in

Set breakpoint to receive notification via debugger whenever entering the FUN_00414550 function of interest.

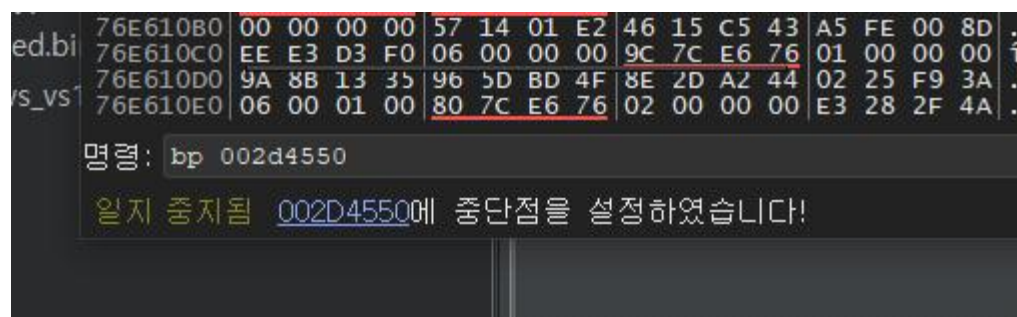


Figure 35 - set break point at interesting function's offset address

The values returned from the function of interest FUN_00414550 are stored in the EAX buffer, as shown in Figure 36 and 37 below. That's not very interesting information. The breakpoint is then indicated by an argument of 'jhl#A' and somethings.

The screenshot shows the Ghidra decompiler interface for the function FUN_00414550. The assembly view on the left shows instructions from 002D4550 to 002D45A8. The right pane shows the decompiled code, where the return value is stored in EAX (002C1020) and is labeled 'j hl#/'. The status bar at the bottom indicates the current instruction is 002D4550, which is 'push ebp'.

Figure 36 - First value returned from FUN_00414550 function

The screenshot shows the Ghidra decompiler interface for the function FUN_00414550 at address 002D4676. The assembly view on the left shows instructions from 002D4676 to 002D46F6. The right pane shows the decompiled code, where the return value is stored in EAX (002F7B74) and is labeled '&63d32d239380a49b6f83128fe71ea01'. The status bar at the bottom indicates the current instruction is 002D4676, which is 'push ebp'.

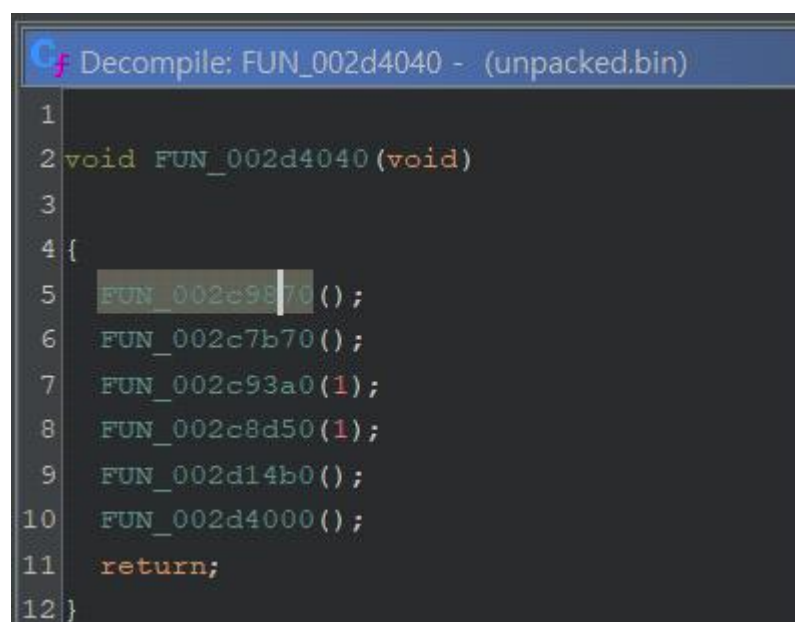
Figure 37 - second value returned from FUN_00414550 function

Eventually I realized that this function wasn't used to decode the final string. Rather, it was to get a copy of the same base64 obfuscated strings that were previously discovered. At this point, I experimented with suspicious imports, but I couldn't reliably trace them to a function that could get decoded C2. While investigating Entry Function in Ghidra, I found 2 interesting function for Amadey major malicious features. This function is located at 002d602a in the main() function and is named '__scrt_common_main_seh()' in Ghidra's auto-decompiler. Inside that function, the FUN_002d4040() function is called, which is located at the address 002d5f98.

The screenshot shows the Ghidra decompiler interface for the function __scrt_common_main_seh() in the main() function. The assembly view on the left shows instructions from 002d5f94 to 002d5fa7. The right pane shows the decompiled code, where the function calls FUN_002d4040(). The status bar at the bottom indicates the current instruction is 002d5f98, which is 'CALL FUN_002d4040'.

Figure 38 - found some interest function in main() function using Ghidra

There were six functions within the FUN_002d4040() function



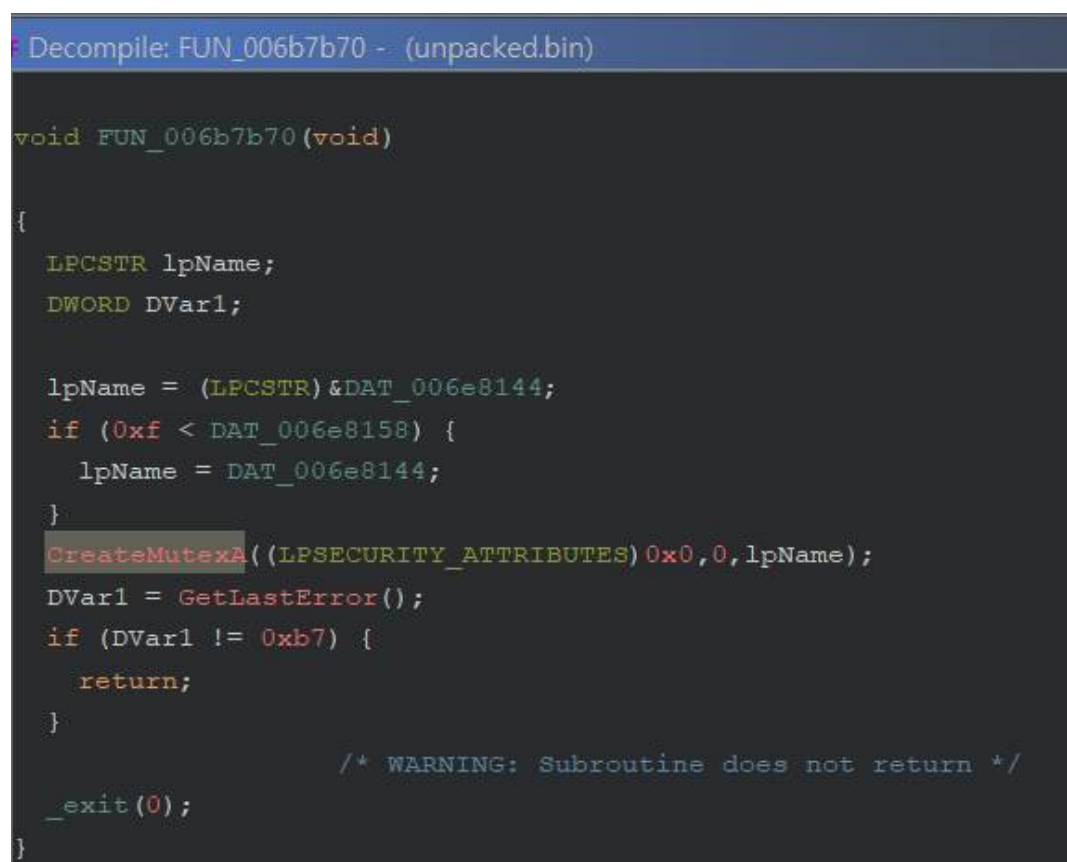
```

Decompile: FUN_002d4040 - (unpacked.bin)
1
2 void FUN_002d4040(void)
3
4 {
5     FUN_002c9870();
6     FUN_002c7b70();
7     FUN_002c93a0(1);
8     FUN_002c8d50(1);
9     FUN_002d14b0();
10    FUN_002d4000();
11    return;
12 }

```

Figure 39 - 6 functions exists at FUN_002d4040() function

In this process, the second FUN_002c7b70() function created Mutex of Amadey and set only one Amadey process to run. After invoking 'GetLastError' WinAPI function, there is a procedure to check if the value of DVar1 is equal to 0xb7. In Figure40, 0xb7 means that Mutex has already been created. If the mutex creation returned a value of 0xb7 (Already Exists) - then the malware would terminate itself.



```

Decompile: FUN_006b7b70 - (unpacked.bin)

void FUN_006b7b70(void)
{
    LPCSTR lpName;
    DWORD DVar1;

    lpName = (LPCSTR) &DAT_006e8144;
    if (0xf < DAT_006e8158) {
        lpName = DAT_006e8144;
    }
    CreateMutexA((LPSECURITY_ATTRIBUTES) 0x0, 0, lpName);
    DVar1 = GetLastError();
    if (DVar1 != 0xb7) {
        return;
    }

    /* WARNING: Subroutine does not return */
    _exit(0);
}

```

Figure 40 - Creating and check mutex feature in Amadey bot

While investigating FUN_002d4040(), I identified a function with interest. The function is FUN_002c9870 and is located at 002c9870.

Figure 41 below focuses on specific lines defined in FUN_002c9870. The address value of the line is 002c98b2 and is considered to be related to the contents confirmed in Figures 36 and 37.

As can be seen in Figures 36 and 37, the FUN_002c4550 function is used to obtain the obfuscated base64 string stored in the .rsrc section of the file. Although the FUN_002c4550 function does not directly decrypt, it has been confirmed that the obfuscated base64 strings returned through the FUN_002c9870 function are decrypted in plain text as they pass through the FUN_002c98b2 function.

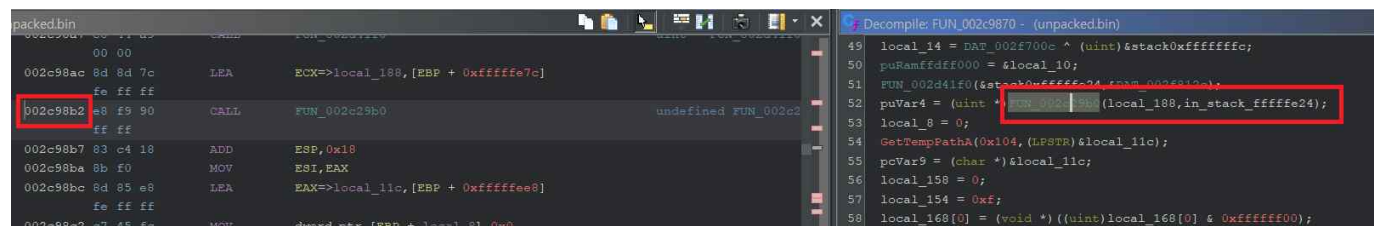


Figure 41 - finding suspected function(actual decode func) using Ghidra

I tried to set breakpoint to 002c98b2.

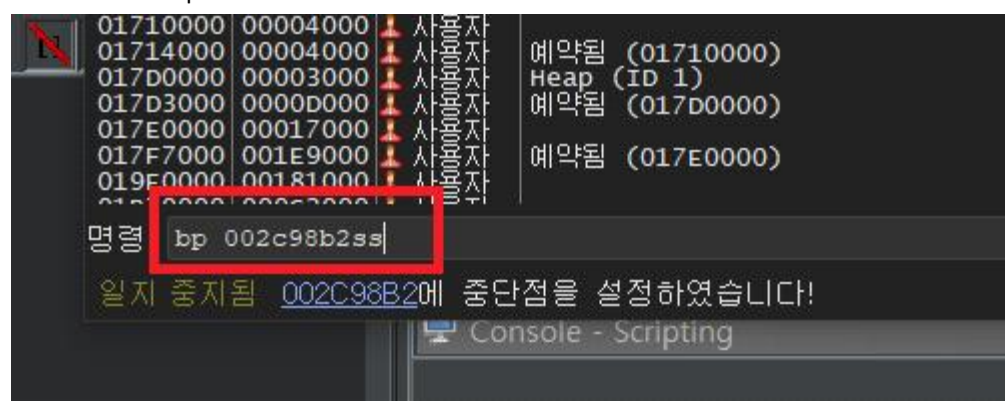


Figure 42 - set breakpoint at suspicious offset

It was confirmed that FUN_002c98b2 was called, breakpoint was stuck in x32Dbg, and obfuscated base64 string was entered in the EAX register buffer.

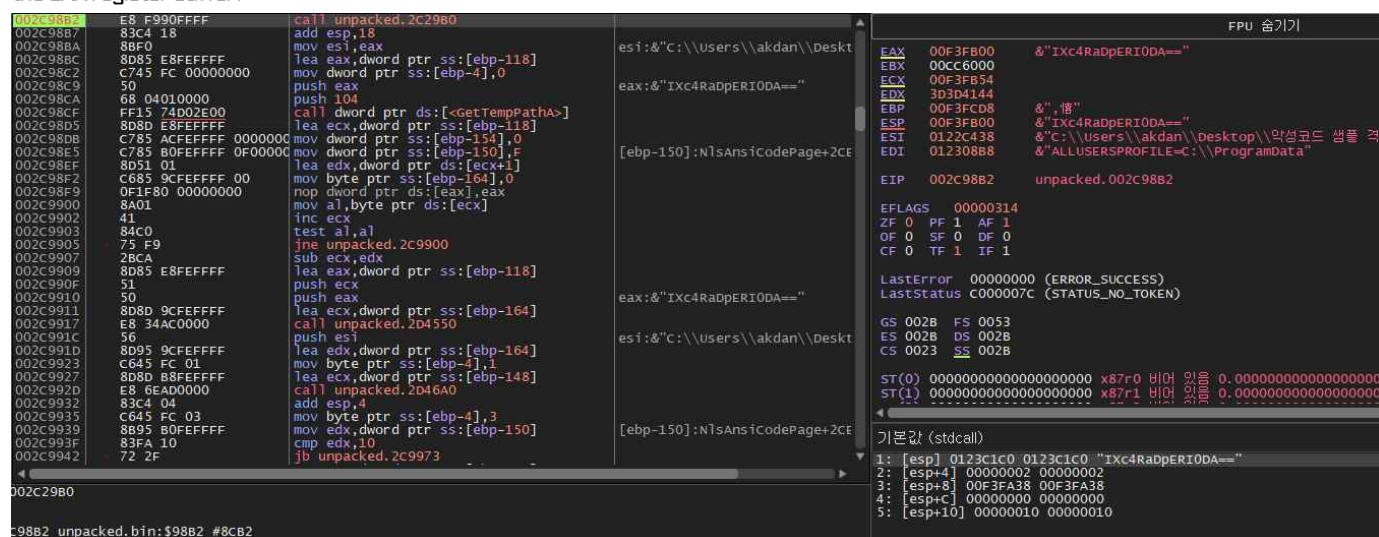


Figure 43 - check suspicious function input using x32Dbg

As shown at Figure 44, it was confirmed that the obfuscated string input at Figure 43 was decrypted and returned to the EAX

register buffer in plain text.

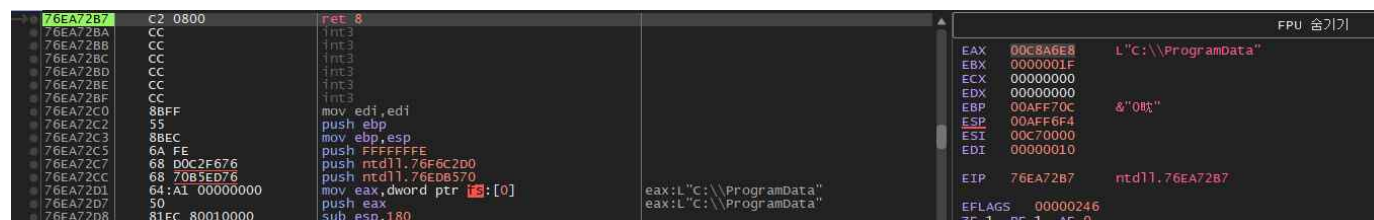


Figure 44 - Decoded string is seen in EAX buffer

execute 'Execute until Return' to allow all of the malicious actions to be executed, the sample dropped a file such as Figure 45 The dropped file path is **"C:\Users\{USER_NAME}\AppData\Local\Temp\59f021478"**

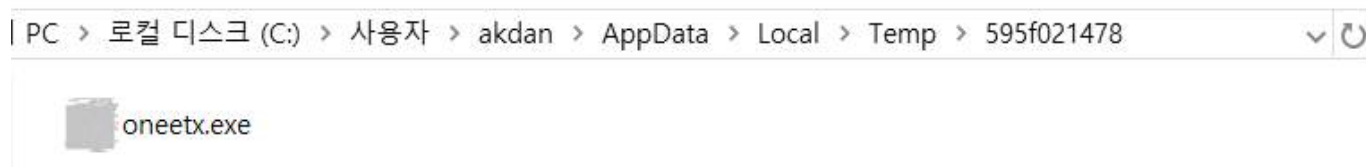


Figure 45 - dropped file

When Ghidra checked the function call tree in the FUN_006b9870 function, it was found to call WinAPI such as **'GetTempPathA'**, **'GetModuleFileNameA'**, and **'CreateDirectoryA'**. Each time an unpacked sample file was loaded, an Int 29 (exit code) was called by an undefined function other than Mutex and the program ended due to Amadey decodes some of the hard-coded obfuscated strings containing the path in which it will be installed. and uses them as a kind of anti-reversing something. Kinda of this contents confirmed at Figure 51.

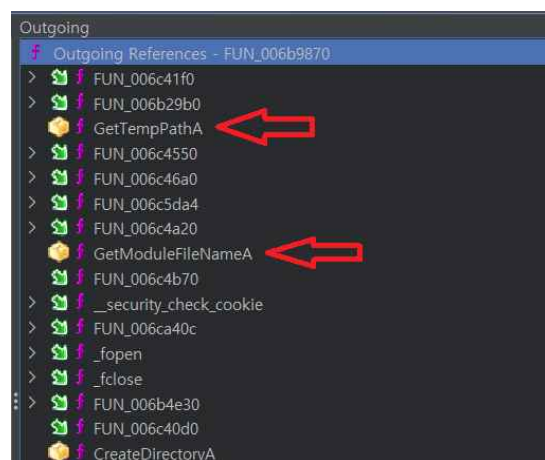


Figure 46 - result of FUN_006b9870's Function call tree

Updated Ghidra's default address to match the new address of x32dbg after the new file was loaded.

주소	크기	파티	정보	내용	유형	보호	초기
006B0000	00001000	사용자	oneetx.exe		IMG	-R---	ERWC-
006B1000	00002C00	사용자	".text"		IMG	ER---	ERWC-
006DD000	0000A000	사용자	".rdata"		IMG	-R---	ERWC-
006E7000	00003000	사용자	".data"		IMG	-RW--	ERWC-
006EA000	00001000	사용자	".rsrsc"		IMG	-R---	ERWC-
006EB000	00003000	사용자	".reloc"		IMG	-R---	ERWC-
00B20000	00010000	사용자			MAP	-RW--	-RW--

Figure 47 - find base image address from dropped file when loaded at x32dbg

Also set Ghidra syncs the base image address of the newly loaded file to x32Dbg

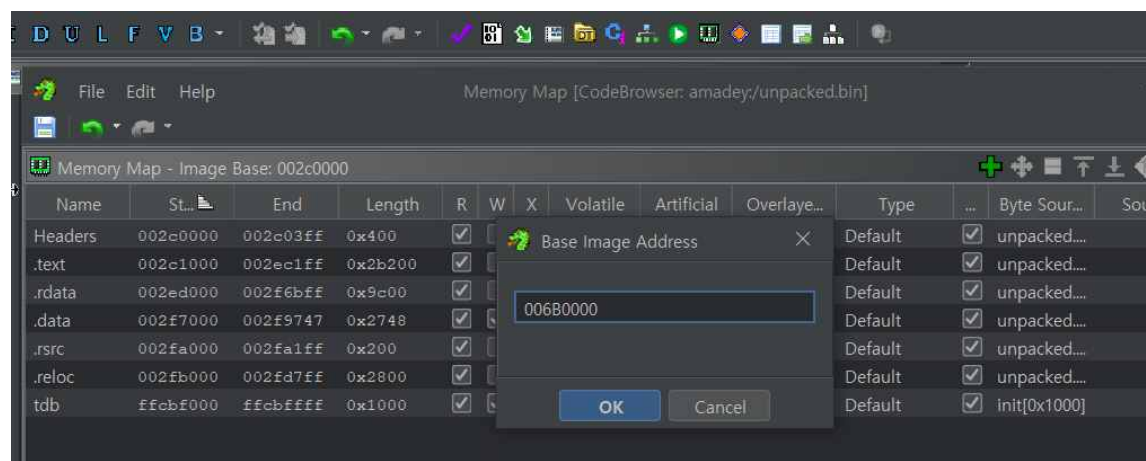


Figure 48 - Sync to x32Dbg base image address

The attachment to Figure 49 below contains the process of Amadey decrypting the name of the folder in which it will be installed from the obfuscated string.

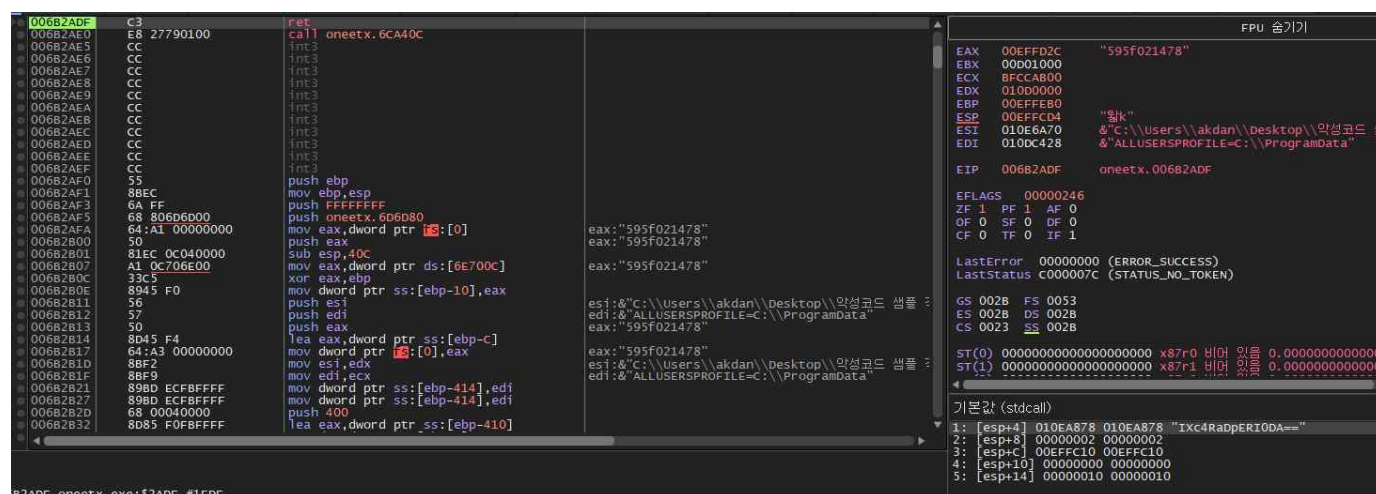


Figure 49 - result of install folder name from FUN_002c98b2 at sample

While analyzing this FUN_002c98b2 function, I found an interesting value, '76XoROUVDkN1QQ==' stored in the EAX register.

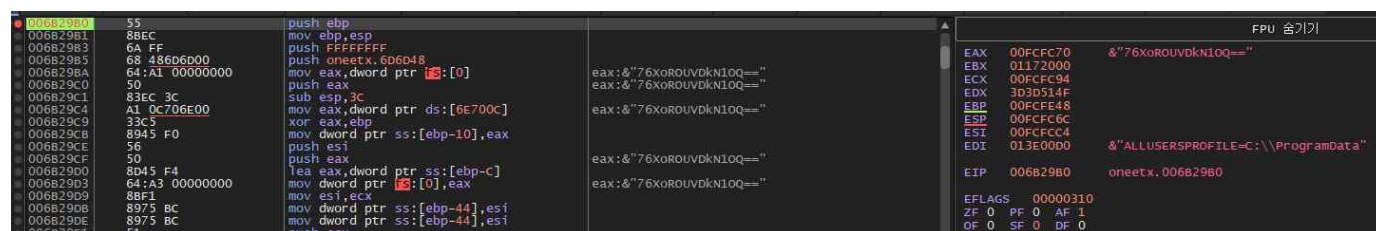


Figure 50 - interesting value entered at EAX buffer from FUN_002c98b2

The '76XoROUVDkN1QQ==' stored in the EAX value passes through the FUN_002c98b2 function and is returned with the plaintext string 'oneetx.exe'. As a result of cross-checking, it was confirmed that the initial analyzed sample 'si684017.exe' and the dropped sample 'oneetx.exe' currently being analyzed by x32Dbg go through the same process as Figure 51.

The initial analysis sample 'si684017.exe' checks whether the corresponding path and file exist when this process is over. The

dropped sample 'oneetx.exe' checks whether the path it is running is the same as the combined path stored in EAX, EDX, and after this process. If the path combined with the execution path does not match, only the initial installation process (startup registration, scheduler registration) is performed and no additional malicious behavior is performed.

In this process, it was determined that the time when the corresponding decryption function was called was to decompress the payload and drop it into the system, or to apply it as an analysis prevention technology for the dropped file.

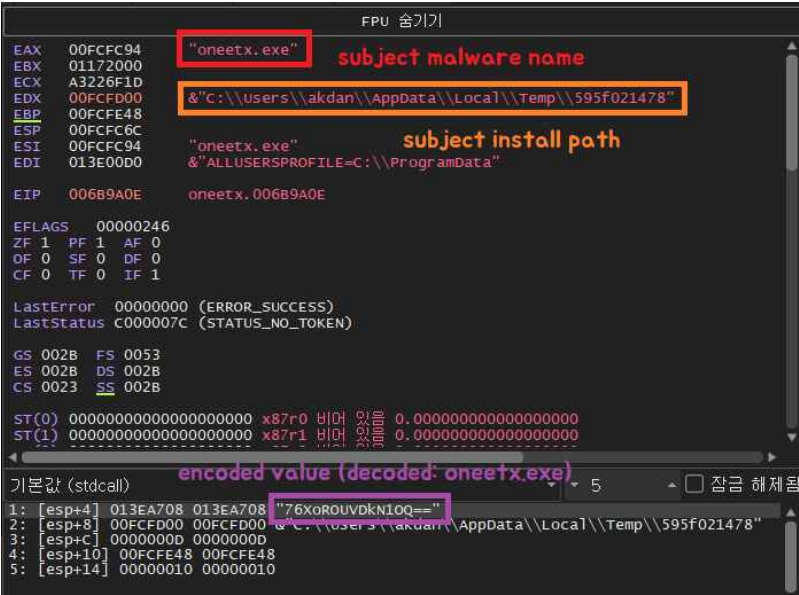


Figure 51 - Amadey bot tried to combine install path from encoded strings

While analyzing the function, I found a combination of the C2 address that Amadey needed in the process of attempting to install additional plugins. The C2 address identified as in Figure52, the address is 77.91.124.207 and is attempting to install an additional malicious plugin named cred64.dll.

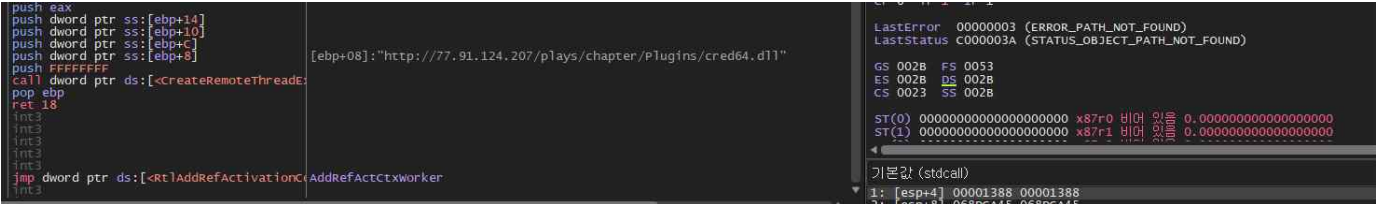


Figure 52 - identifying amadey requests download cred64.dll plugin from C2 server

PC > 로컬 디스크 (C:) > 사용자 > akdan > AppData > Roaming > 006700e5a2ab05			
이름	수정한 날짜	유형	크기
clip64.dll	2025-06-21 오후 8:20	응용 프로그램 확장	0KB
cred64.dll	2025-06-14 오전 2:19	응용 프로그램 확장	0KB

To automate character string decryption, we tried manually patching Mutex's value to 86. Originally, breakpoint was activated on FUN_002c98b2 after a line to compare and verify Mutex's value, decrypting base64 like strings with all the unknown additional encryption modules present in the sample, and extracting them all with the conditional midpoint setting of x32Dbg. In my case, the offset address is 00407B94, which is for check Mutex.

However, after performing a file path inspection detour and a Mutex detour for some reason, Amadey fell into the Sleep loop. Unfortunately, we could not go further in this process, so we had no choice but to extract only the information up to this point.

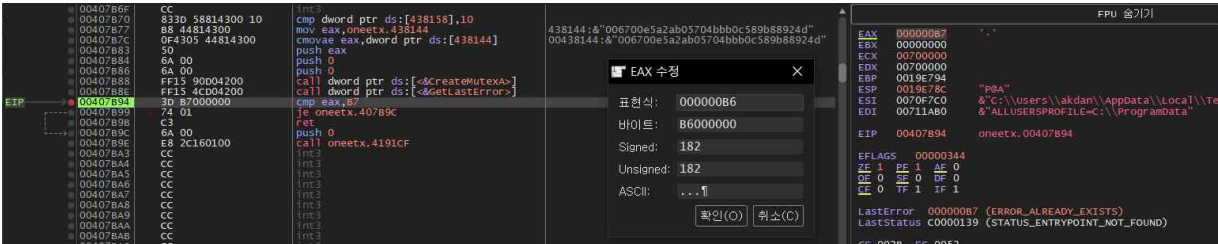


Figure 53 - Fix 00407B94 offset's value for bypass Mutex check

77.91.124.207

Ports

- 80 TCP

WHoIs

inetnum: 77.91.124.0 - 77.91.124.255
descr: H2.NEXUS Frankfurt Network
abuse-c: HL5106-RIPE
remarks: H2.NEXUS Frankfurt Network
geofeed: https://h2.nexus/media/geofeed.csv
geoloc: 50.0379 8.5622
netname: H2NEXUS
country: DE
admin-c: HL5106-RIPE
tech-c: HL5106-RIPE
status: SUB-ALLOCATED PA
mnt-by: NEXUS-MNT
created: 2024-07-11T20:06:15Z
last-modified: 2024-10-15T02:24:59Z
source: RIPE
role: H2NEXUS LTD
address: 71-75, Shelton Street, Covent Garden, London, United Kingdom, WC2H 9JQ
abuse-mailbox: abuse@h2.nexus
nic-hdl: HL5106-RIPE

Recommendations

Author recommends that users and administrators consider using the following best practices to strengthen the security posture of their organization’s systems. Any configuration changes should be reviewed by system owners and administrators prior to implementation to avoid unwanted impacts. The following checklist reflects the author’s personal best-practice recommendations for safeguarding endpoints and servers. Review and test any configuration change in a lab or staging environment before applying it to production systems.

- Maintain up-to-date antivirus signatures and engines.
- Keep operating system patches up-to-date.
- Disable File and Printer sharing services. If these services are required, use strong passwords or Active Directory authentication.
- Restrict users’ ability (permissions) to install and run unwanted software applications. Do not add users to the local administrators group unless required.
- Enforce a strong password policy and implement regular password changes.
- Exercise caution when opening e-mail attachments even if the attachment is expected and the sender appears to be known.
- Enable a personal firewall on agency workstations, configured to deny unsolicited connection requests.

- Disable unnecessary services on agency workstations and servers.
- Scan for and remove suspicious e-mail attachments; ensure the scanned attachment is its "true file type" (i.e., the extension matches the file header).
- Monitor users' web browsing habits; restrict access to sites with unfavorable content.
- Exercise caution when using removable media (e.g., USB thumb drives, external drives, CDs, etc.).
- Scan all software downloaded from the Internet prior to executing.
- Maintain situational awareness of the latest threats and implement appropriate Access Control Lists (ACLs).

Additional information on malware incident prevention and handling can be found in National Institute of Standards and Technology (NIST) Special Publication 800-83, "**Guide to Malware Incident Prevention & Handling for Desktops and Laptops**".

Contact Information

- Github (github.com/miho030)

Document FAQ

Q. Is it safe to deploy the IOCs from this report directly in a live environment?

A. Best practice is to apply the IOCs in a test or staging environment first, verify there are no false positives, and then roll them out to production in phases.

Q. May I redistribute this report for commercial purposes?

A. This report may be freely shared for non-commercial and public-interest use only. Creating commercial derivative works requires explicit permission from the author.

Q. Where can I obtain malware samples for my own analysis?

A. You can legally download samples from public repositories and sandboxes such as VirusTotal, MalwareBazaar, and ANY.RUN. If you execute them locally, always use an isolated analysis environment (e.g., a virtual machine or Cuckoo sandbox).